StaffML: A Physics-Grounded Interview Question Bank for Machine Learning Systems Engineers

System Design, Competency Model, and Quality Infrastructure

Vijay Janapa Reddi
Harvard University
vj@eecs.harvard.edu

April 3, 2026

Abstract

We present StaffML, a curated bank of 9,199 interview questions for ML systems engineers spanning 5 deployment tracks and 6 mastery levels. Every question is classified on four independent axes: *topic* (87 concepts across 13 areas with 57 prerequisite edges), *zone* (11 cognitive modes capturing *how* a topic is tested), *track* (physics regime), and *level* (cognitive depth). Unlike existing resources focused on algorithmic coding, StaffML tests *mechanical sympathy*—the ability to reason quantitatively about physical constraints of ML infrastructure. Every question requires napkin math grounded in real hardware specifications. The key contribution is the *ikigai competency model*, where four fundamental skills (recall, analyze, design, quantify[1]) combine pairwise into six compound cognitive zones (e.g., diagnosis, specification, fluency) and unite into a mastery zone, providing a structured vocabulary for ML systems engineering expertise. The scope is deliberately focused on *technical systems reasoning*, orthogonal to behavioral or leadership competencies. The classification schema generates schemas from a single source of truth, and quality is rigorously enforced by a 19-check invariant system, exhaustive multi-model math verification, and three-stage deduplication.

1 Introduction

The role of the ML systems engineer has emerged as one of the most sought-after positions in the technology industry. Unlike ML researchers who design model architectures or software engineers who write application code, ML systems engineers must reason about the *physical constraints* of deploying machine learning at scale: memory bandwidth limits that determine whether a workload is compute-bound or memory-bound, thermal envelopes that cap sustained throughput on edge devices, network congestion that dominates training time in distributed clusters, and the economics of accelerator fleets where a single misconfigured batch size can waste thousands of dollars per hour.

This is what Martin Thompson calls *mechanical sympathy* (Thompson, 2011) — a term borrowed from racing driver Jackie Stewart's philosophy that a driver performs best when they understand what the car is doing underneath them. A Formula 1 driver who understands the engine does not merely drive faster; they drive *differently*, making decisions that are physically impossible to arrive at through pure algorithmic thinking. Similarly, an ML systems engineer who understands that autoregressive decode reads the full 140 GB model weights for every single token — achieving an arithmetic intensity of just 1 FLOP/Byte, far below the hardware's 148 FLOP/Byte ridge point — does not merely optimize code; they architect systems that batch requests to amortize the bandwidth cost.

The growing field of AI engineering demands practitioners who can reason about safety, efficiency, robustness, and reliability — not just model accuracy. StaffML assesses these systems-level competencies that distinguish engineers who can deploy ML responsibly at scale.

In an era where large language models can seamlessly generate code, reverse linked lists, and pass algorithmic screens (OpenAI, 2023), the traditional coding interview has lost its predictive power. For Staff+ hiring, algorithmic fluency is no longer a sufficient signal. What remains defensible is *mechanical sympathy*—the ability to reason from first principles about the physical constraints of ML infrastructure. The answer is that *you cannot prompt your way out of a memory wall*. No amount of LLM-assisted coding will tell you that your serving cluster is memory-bandwidth-bound at an arithmetic intensity of 1 FLOP/Byte, or that

[1] We use quantify" rather than implement" to emphasize producing concrete numbers over writing production code.

your KV-cache will consume 42 GB at 128K context, or that thermal throttling will cut your edge device's sustained throughput by 40%. These are physics problems, and physics requires quantitative reasoning grounded in real hardware specifications.

We use the term *physics-grounded* to mean that every question has a numerical answer derivable from real hardware specifications: memory bandwidths, thermal envelopes, interconnect latencies, and power budgets. The companion MLSysIM simulator (Reddi, 2026b) provides the shared physics engine from which all hardware constants are drawn, ensuring that questions are numerically consistent and verifiable. A candidate who internalizes these physical constants can derive answers for any model on any hardware, rather than memorizing configuration recipes that expire with each hardware generation.

Yet interview preparation resources for this role are remarkably thin. LeetCode (LeetCode Inc., 2024) offers 2,800+ algorithmic coding problems but nothing on GPU memory hierarchies or KV-cache sizing. Huyen's *Designing Machine Learning Systems* (Huyen, 2022) provides excellent conceptual coverage but no interactive, quantitative practice (see Section 13 for a detailed comparison). Mock interview platforms focus on behavioral and coding rounds, not system design with napkin math.[2] The result is a preparation gap: candidates study the wrong material, and interviewers lack a validated question bank calibrated to the specific cognitive demands of systems reasoning.

> **Example (diagnosis zone, cloud track, L4):** Your production LLM serving cluster reports 40% MFU during the decode phase. The H100s are rated at 495 TFLOPS FP16 dense. Your team assumes the GPUs are underclocked. Diagnose the actual root cause.
>
> *Common mistake:* Blaming hardware or driver issues.
>
> *Napkin math:* (1) Decode reads the full model weights per token: $2 \times 70\text{B} = 140$ GB in FP16. (2) Compute per token: $2 \times 70\text{B} = 140$ GFLOPs. (3) Arithmetic intensity $= 140$ GFLOPs $\div$ 140 GB $= 1$ FLOP/Byte. (4) H100 ridge point $= 495$ TFLOPS $\div$ 3.35 TB/s ≈ 148 FLOPs/Byte.
>
> *Answer:* The decode phase is memory-bandwidth-bound. At arithmetic intensity of 1 FLOP/Byte, the workload sits far below the 148 FLOP/Byte ridge point. MFU measures compute utilization, but the bottleneck is data movement. The 40% figure is not a deficiency — the GPUs are performing as expected.

The intellectual foundation of STAFFML is the thesis that **constraints drive architecture**. You do not choose a Transformer because it is trendy; you choose it because its attention mechanism parallelizes across thousands of cores on real silicon, unlike the sequential dependency chains of RNNs. You do not use mixed precision because a tutorial told you to; you use it because FP16 tensor cores deliver 2× the throughput of FP32 and halve the memory footprint, changing whether your model fits on one GPU or requires four. This philosophy, drawn from the *Machine Learning Systems* textbook (Reddi, 2026a), insists that AI is not magic; it is infrastructure, and infrastructure has laws.

The constraints that drive ML systems architecture can be distilled into five fundamental laws:

1. **The Roofline Bound.** Every workload is either compute-bound or memory-bandwidth-bound, determined by its arithmetic intensity relative to the hardware's ridge point.
2. **The Memory Capacity Wall.** Model weights, optimizer state, activations, and KV-cache must fit simultaneously in available memory; exceeding capacity forces parallelism, offloading, or compression.
3. **The Communication Tax.** In distributed systems, gradient synchronization, tensor transfers, and pipeline bubbles consume time proportional to model size and inversely proportional to interconnect bandwidth.
4. **The Power Envelope.** Sustained throughput is bounded by thermal dissipation, not peak compute; real performance is the minimum of compute capability and cooling capacity.
5. **The Cost Multiplier.** Every architecture decision compounds through fleet size, electricity cost, and amortization period; a 2× efficiency improvement at 1,000 GPUs saves millions of dollars annually.

Every STAFFML question ultimately tests reasoning about one or more of these laws. The topic taxonomy maps concepts to the laws they exercise; the zone axis determines how that reasoning is assessed.

STAFFML fills this gap with thousands of questions that test mechanical sympathy across the full ML deployment spectrum. Every question has a realistic interview scenario drawn from industry practice, a common mistake that candidates typically make, a realistic solution with quantitative reasoning, napkin math grounded in real hardware specifications, and classification by topic (what concept), zone (how it is tested), track (where it deploys), and level (how hard it is). STAFFML is the companion assessment tool for the *Machine Learning Systems* textbook (Reddi, 2026a), developed alongside the Harvard CS249r course (Reddi, 2024).

Three practical workflows. STAFFML supports three practical workflows:

1. **Interview preparation.** Candidates filter questions by target role level (L4–L6+), practice zone, and deployment track. Depth chains provide structured study

[2] *Napkin math* (back-of-envelope calculation) is rapid quantitative estimation from first principles. For example: "How much VRAM does Llama-70B's KV-cache need at 128K context?" $\rightarrow 2 \times 80 \times 8 \times 128 \times 128\text{K} \times 2 \approx 42$ GB. The candidate derives the answer in real time from five numbers: layers, KV-heads, head dimension, sequence length, and bytes per element.

paths from fundamentals to synthesis (see Section 3.5 for depth chains that structure study paths).

2. **Interview design.** Hiring managers select questions across zones to construct balanced interview loops. The zone model provides a shared vocabulary for debrief — "strong in diagnosis, weak in specification" (Section 12 describes interview loop design).
3. **Curriculum alignment.** Educators map course content to the topic taxonomy, identifying which competency cells their curriculum covers and where gaps remain (the prerequisite graph in Section 4 suggests teaching order).

Backward design. We employ a *backward design* methodology (Wiggins and McTighe, 2005), detailed in Section 2, to derive the assessment structure before constructing questions. The intersection of five constraints (competency decomposition, topic curation, physics applicability, capacity bounds, and quality invariants) yields a principled corpus where every question traces back to foundational engineering competencies. Section 11 presents example questions. Section 12 discusses practical interview design. Section 13 surveys related work. Section 14 concludes with future directions.

2 Methodology: Backward Design

The structure of StaffML is not an organizational convenience; it is a derivation. We follow the *backward design* methodology of Wiggins and McTighe (Wiggins and McTighe, 2005), which proceeds in three stages: identify the desired results, determine what constitutes acceptable evidence, and then—only then—design the assessment. The corpus size is an *output* of this derivation, not an input. We did not begin with a target of N questions and fill to quota; we defined the competency space, filtered it by physics, bounded its capacity empirically, and let the total fall out.

Figure 1 illustrates the five-step derivation chain. Each step constrains the next, and the final corpus is fully determined by the intersection of those constraints.

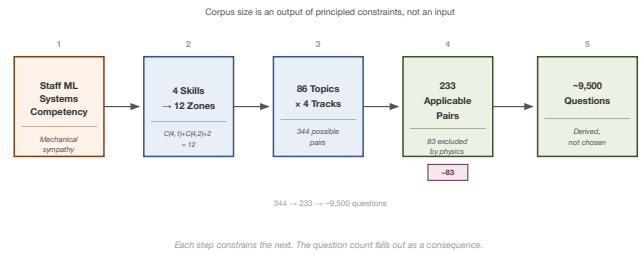

Figure 1. The backward design derivation chain. Corpus size is an output of principled constraints, not an input. Each step narrows the assessment space: competency decomposition yields 11 zones, topic curation yields 87 concepts, the physics filter excludes impossible hardware pairings, and capacity analysis bounds meaningful question density.

2.1 The Derivation Chain

Step 1. Decompose competency into skills and zones. We begin from the desired result: *what must a Staff-level ML systems engineer demonstrate?* The answer is mechanical sympathy—the ability to connect hardware constraints to architectural decisions using quantitative reasoning. We decompose this into four atomic skills (recall, analyze, design, quantify) and observe that real engineering tasks never exercise one skill in isolation. A diagnosis requires recall *and* analysis; napkin math requires recall *and* quantification. The pairwise combinations of four skills yield six compound zones, plus one mastery zone that exercises all four. This produces the 11 cognitive zones described in Section 3.

The zone model complements Bloom's revised taxonomy (Anderson et al., 2001). While Bloom's two-dimensional framework (cognitive process $\times$ knowledge type) captures the *depth* of cognitive engagement, it does not capture the *simultaneous exercise* of multiple cognitive modes that characterizes real engineering tasks. Consider fluency (recall $\cap$ quantify): an engineer who retrieves hardware specifications from memory *and* executes napkin math in real time. Bloom has no single level for this act. The ikigai model treats skills as *dimensions*, not *levels*. The zones are the cells of a combinatorial lattice: $\binom{4}{1} + \binom{4}{2} + 1 = 11$.

Step 2. Define topic coverage. We identify 87 topics spanning 13 competency areas as the minimum spanning set of Staff-level ML systems knowledge. The prerequisite graph (57 edges) encodes dependencies.

Step 3. Apply the physics filter. Not every topic has a physical substrate on every hardware tier. We derive an *applicability matrix* that excludes 83 topic–track pairs with one-sentence physics justifications. The remaining 233 pairs define the valid assessment space. This establishes *content validity* (Messick, 1995): every question references a concept with physical meaning on the targeted hardware tier.

Step 4. Bound capacity empirically. For each valid cell, how many meaningfully distinct questions exist? Capacity varies by cognitive complexity and diffi-

culty level. These bounds are hypotheses validated through expert enumeration and planned saturation studies.

Step 5. Validate through convergence. We track whether expert reviewers identify new structural gaps across rounds. When feedback shifts from "missing X" to "improve phrasing of Y," the framework is structurally converged. Twelve domain experts reviewed the framework; the structure stabilized after corrections to the applicability matrix and addition of consensus topics.

Construct validity—whether zones capture psychometrically distinct competencies—requires inter-rater reliability data (Cohen's $\kappa > 0.7$) and Item Response Theory (Lord, 1980) calibration from candidate response data. Both are planned as future work (Section 14).

3 The StaffML Competency Model

What makes ML systems expertise different from algorithmic programming skill? A software engineer who can implement a Transformer from scratch (Reddi, 2026c) may still be unable to answer a deceptively simple question: "How much VRAM does that model need, and why does that number determine your entire serving architecture?" The gap is not knowledge of algorithms; it is *mechanical sympathy* — the ability to reason quantitatively about how software interacts with physical hardware.

The TinyTorch competency matrix (Reddi, 2026c) defines *what to teach* through 8 knowledge areas crossed with 5 capability levels; the StaffML ikigai model defines *how to assess* whether those competencies have been acquired.

StaffML begins from the observation that ML systems expertise is not a single skill but the *intersection* of multiple cognitive modes. An engineer who can recall every hardware specification but cannot analyze a failure is incomplete. An engineer who designs elegant architectures but cannot produce napkin math to validate them is dangerous. The competency model must capture these intersections explicitly.

3.1 Four Fundamental Skills

We decompose ML systems expertise into four fundamental skills. The model is inspired by the Japanese concept of *ikigai*[3] — the intersection of overlapping life domains — adapted here as four skill domains whose intersections define qualitatively different cognitive acts.

Every act of ML systems reasoning exercises one or more of these skills:

1. **Recall.** Facts, definitions, hardware specifications. "What is the HBM bandwidth of an H100?" The engineer retrieves a stored constant.
2. **Analyze.** Tradeoffs, reasoning, root cause analysis. "Why is this workload memory-bound?" The engineer reasons about the interaction between compute and data movement.
3. **Design.** Architecture decisions, requirements-to-system mapping. "Design a serving stack with P99 $<$ 100 ms." The engineer translates constraints into an architecture.
4. **Quantify.** Napkin math & estimation. "Calculate the VRAM required for a 70B model at 128K context." The engineer produces a specific number from a formula and hardware specs.[4]

These four skills are not Bloom's levels; they are orthogonal dimensions of expertise. A senior engineer may have deep recall and strong quantification skills (fluency) but weak design ability; another may excel at analysis and design (evaluation) but struggle with napkin math. The ikigai model makes these profiles visible and assessable.

3.2 Eleven Cognitive Zones

The four skills combine into 11 zones: 4 pure zones (exercising a single skill), 6 compound zones (exercising two skills simultaneously), and 1 mastery zone (all four). Figure 2 illustrates the model as a four-ellipse Venn diagram. We deliberately omit the four possible three-skill intersections (e.g., recall $\cap$ analyze $\cap$ design) because, in practice, questions exercising three skills almost always exercise all four, making them mastery questions. The two-skill compound zones represent the cognitively distinctive acts; adding three-skill zones would dilute rather than sharpen the classification.

Pure zones test a single skill in isolation:

- **Recall.** "What is the ridge point of an H100?" Retrieve a fact.
- **Analyze.** "Why does GPU utilization collapse at batch size 1?" Reason about system behavior.
- **Design.** "Architect a KV-cache paging system." Produce an architecture.
- **Quantify.** "Write the formula for KV-cache memory." Produce a concrete artifact.

Compound zones test two skills simultaneously. These are the zones that most sharply differentiate strong from weak candidates, because they require integrating knowledge across cognitive modes:

[3] *Ikigai* is a Japanese concept describing the intersection of passion, skill, need, and reward. We adapt its four-circle Venn diagram structure to model intersecting cognitive skills in ML systems engineering.

[4] We use "quantify" rather than "implement" to emphasize that this skill tests the ability to produce concrete numbers from specifications, not the ability to write production code. The distinction matters: quantification is a reasoning act (estimating, bounding, computing), whereas implementation implies building artifacts.

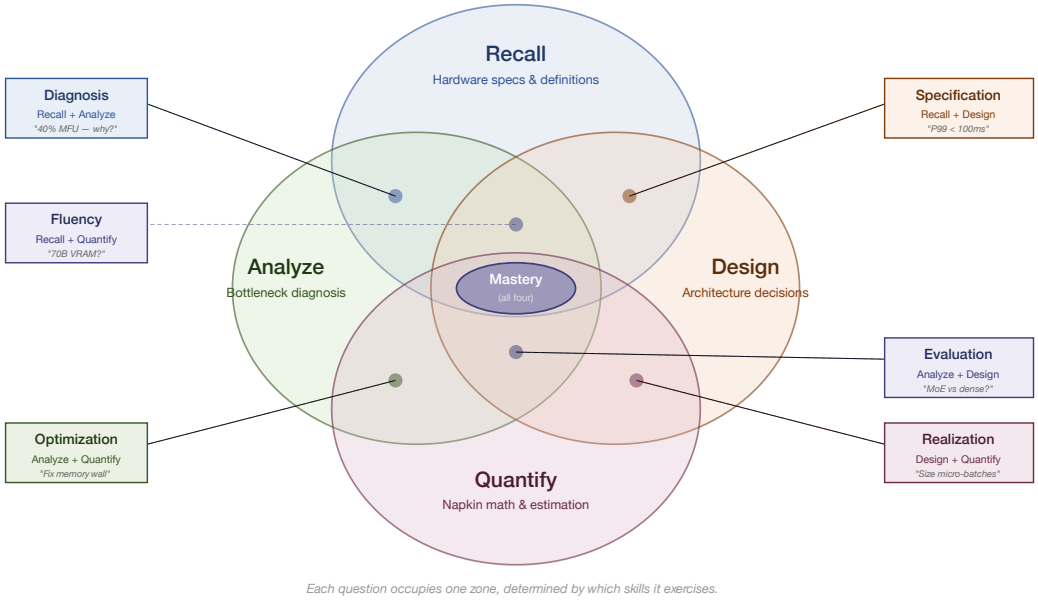

Figure 2. The STAFFML ikigai competency model. Four fundamental skills (Recall, Analyze, Design, Quantify) overlap to produce six compound zones and one central mastery zone. Each question targets exactly one zone, which determines *how* a topic is assessed. The compound zones capture the cognitive acts that distinguish Staff-level engineers: diagnosing failures (Recall ∩ Analyze), translating specifications into architectures (Recall ∩ Design), performing napkin math from memory (Recall ∩ Quantify), evaluating architectural alternatives (Analyze ∩ Design), sizing concrete systems (Design ∩ Quantify), and optimizing bottlenecks (Analyze ∩ Quantify).

Table 1. Zone definitions. The 11 ikigai zones with constituent skills, one-sentence descriptions, and example question stems.

Zone	Skills	Description	Example Question Stem
Recall	recall	Retrieve facts, definitions, and hardware specifications from memory.	"What is the HBM bandwidth of an H100?"
Analyze	analyze	Reason about system behavior, tradeoffs, and root causes.	"Why does GPU utilization collapse at batch size 1?"
Design	design	Make architecture decisions and map requirements to systems.	"Architect a KV-cache paging system for variable-length sequences."
Quantify	quantify	Produce concrete numbers from formulas and hardware specifications.	"Write the formula for KV-cache memory consumption."
Diagnosis	recall + analyze	Identify root causes from observable symptoms using hardware knowledge.	"MFU is 40% during decode. Diagnose it."
Specification	recall + design	Translate quantitative constraints into a concrete system architecture.	"P99 TTFT $<$ 200 ms for a 70B model. Design the serving stack."
Fluency	recall + quantify	Perform napkin math from memory, recalling both formulas and constants.	"How much VRAM does Llama-70B need at 128K context?"
Evaluation	analyze + design	Analyze tradeoffs and select the best architecture for given constraints.	"MoE vs. dense for your latency budget: which and why?"
Realization	design + quantify	Turn architecture decisions into concrete numbers and schedules.	"Size the micro-batches for this pipeline-parallel training job."
Optimization	analyze + quantify	Diagnose a bottleneck and calculate the impact of each optimization step.	"Inference is memory-bound at 30% MFU. Walk the optimization ladder."
Mastery	all four	Full Staff-level synthesis requiring recall, analysis, design, and quantification.	"Design a training cluster for a 70B model. Budget compute, memory, network, and cost."

- **Diagnosis** (Recall ∩ Analyze). "MFU is 40%, diagnose it." The candidate must recall hardware specifications *and* reason about which bottleneck explains the gap. This is the "3 AM on-call" skill.
- **Specification** (Recall ∩ Design). "P99 $<$ 100 ms with 70B model, design it." The candidate must recall hardware capabilities *and* translate the latency constraint into an architecture (tensor parallelism degree, batch size, KV-cache budget).
- **Fluency** (Recall ∩ Quantify). "How much VRAM does a 70B model need?" The candidate must recall the formula and hardware specs *and* execute the arithmetic.

Table 2. Zone–level affinity. Pure zones tend toward lower Bloom's levels; compound zones span mid-to-senior; mastery is Staff+.

Zone	Skills	Typical Levels
Recall	recall	L1–L2
Quantify	quantify	L2–L3
Fluency	recall + quantify	L2–L3
Analyze	analyze	L3–L4
Diagnosis	recall + analyze	L3–L4
Design	design	L4–L5
Specification	recall + design	L4–L5
Optimization	analyze + quantify	L4–L5
Evaluation	analyze + design	L5–L6+
Realization	design + quantify	L5–L6+
Mastery	all four	L6+

This is napkin math from memory, the signature skill of Staff+ engineers.

- **Evaluation** (Analyze ∩ Design). "MoE vs. dense: which architecture for this serving workload?" The candidate must analyze the tradeoffs *and* reason about which design better satisfies the constraints. No recall is required; the specs are given.
- **Realization** (Design ∩ Quantify). "Size the micro-batches for this pipeline-parallel training job." The candidate must design the pipeline schedule *and* produce concrete numbers (batch splits, memory per stage, bubble fraction).
- **Optimization** (Analyze ∩ Quantify). "Fix the memory wall." The candidate must analyze the bottleneck *and* produce a concrete fix (kernel fusion, quantization, offloading) with measurable improvement.

Mastery (all four skills) represents full Staff-level synthesis: "Your LLM serving cluster's TTFT spiked from 100 ms to 4 s after a traffic surge. Diagnose the root cause, design a fix, size the required resources, and estimate the cost." This zone requires recalling hardware specs, analyzing the failure mode, designing an architectural solution, and executing the napkin math to validate it.

3.3 Zone–Level Affinity

Each zone has a natural affinity with certain mastery levels, though the mapping is not rigid:

This affinity is enforced as a soft constraint during question generation. An L1 question tagged as "evaluation" is flagged for review, since evaluation (analyze ∩ design) is cognitively inconsistent with Bloom's "Remember" level.

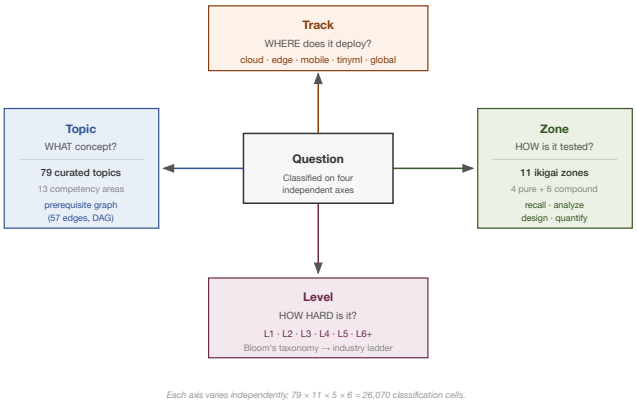

Figure 3. Four-axis classification. Every question is independently classified by *topic* (what concept is tested), *zone* (how the concept is tested), *track* (which physics regime applies), and *level* (what cognitive depth is required). The axes are orthogonal: any valid combination defines a unique assessment cell.

3.4 The Four-Axis Classification

The competency model answers *what cognitive skills matter*. To operationalize this into a practical assessment framework, STAFFML classifies every question on four independent axes, each capturing a distinct dimension of ML systems expertise. Figure 3 illustrates the architecture.

The **topic axis** identifies *what* concept a question tests. The 87 topics are organized into 13 competency areas, from memory hierarchies and parallelism strategies to power budgeting and reliability, and connected by 57 typed edges (prerequisite, broader, narrower, related) that encode the dependency structure of ML systems knowledge.

The **zone axis** captures *how* a concept is tested, using the 11 ikigai zones defined above. The zone determines the cognitive profile of a question independently of its subject matter.

The **track axis** specifies *where* the scenario takes place — which physics regime governs the problem. Cloud questions operate at PFLOPS scale under memory bandwidth and network fabric constraints. Edge questions operate under thermal envelopes and real-time deadlines. Mobile questions face battery budgets and shared-resource contention. TinyML questions confront SRAM limits and hard real-time on bare metal. A fifth "global" track captures cross-cutting principles.

The **level axis** encodes *how hard* the question is, mapped to Bloom's revised taxonomy (Anderson et al., 2001): L1 (Remember) through L6+ (Create). The levels also correspond to industry engineering ladders from intern through Staff+.

These four axes are orthogonal by design. A question about KV-cache management (topic) can test fluency (zone)

in the cloud regime (track) at L3 difficulty (level), or it can test diagnosis in the edge regime at L4. The three-dimensional coverage space of topic × zone × track contains $87 \times 11 \times 4 = 3{,}828$ cells (excluding the cross-cutting global track). The level dimension is not a separate coverage axis; instead, it is captured by the zone–level affinity (Table 2), which determines the natural difficulty range for each zone. Restricting to valid combinations via the applicability matrix (Section 10) yields 2,563 meaningful cells. The combinatorial structure enables precise gap analysis — the coverage space reveals exactly which assessment cells are populated, underfilled, or empty.

3.5 Depth Chains

Questions on the same topic are linked into *depth chains* that progress through Bloom's levels, testing increasingly sophisticated cognitive acts. A chain ensures that assessment covers the full range from recall to synthesis. The corpus contains 979 chains spanning three or more levels.

For example, a chain on KV-cache management progresses through six levels and six zones:

> **L1 (Recall):** "What is a KV-cache and why does it grow with sequence length?"
> **L2 (Quantify):** "Calculate the KV-cache size for Llama-70B at 128K context."
> **L3 (Fluency):** "Your 80 GB GPU OOMs at 128K context. Derive the memory breakdown."
> **L4 (Diagnosis):** "PagedAttention reduces memory by 60%. Explain the mechanism."
> **L5 (Specification):** "Design a KV-cache compression strategy. What precision trade-offs are acceptable?"
> **L6+ (Mastery):** "Your paged KV-cache suffers fragmentation under variable-length batching. Diagnose, design a fix, and size the memory overhead."

Each step in the chain builds on the previous, and the zone shifts from recall through quantification to diagnosis and finally mastery. Chains serve three purposes: (1) validating that the topic taxonomy supports questions at multiple cognitive levels, (2) enabling progressive study paths where candidates build from fundamentals to synthesis, and (3) providing interviewers with calibrated question sequences that reveal the candidate's ceiling.

The corpus contains 979 topic clusters spanning 3+ levels, with 23 full chains spanning all six levels from L1 to L6+. Of the 979 chains, 715 span 3 levels, 104 span 4 levels, 83 span 5 levels, and 109 span all 6 levels. In total, 3,417 questions (64.4% of the corpus) participate in at least one chain.

4 Topic Taxonomy

The topic taxonomy defines *what* each question tests. It organizes the conceptual landscape of ML systems engineering into a structured, navigable knowledge graph. This section describes the curated topics, their prerequisite relationships, and the knowledge organization methodology that governs their selection and maintenance.

Table 3. Competency areas. The 13 areas containing 87 curated topics, organized by systems domain.

Area	Scope	Topics
Architecture	System design patterns	7
Compute	Accelerators, execution models	6
Cross-cutting	Compound AI, MLOps, responsible AI	5
Data	Pipelines, storage, preprocessing	7
Deployment	Serving, compression, inference	7
Latency	Scheduling, prefill/decode, SLOs	6
Memory	VRAM, KV-cache, offloading	8
Networking	Collective ops, interconnects, fabric	6
Optimization	Kernel fusion, graph rewrites, profiling	7
Parallelism	Data/tensor/pipeline/3D parallelism	6
Power	Thermal, energy, battery, DVFS	5
Precision	Quantization, mixed precision, formats	3
Reliability	Fault tolerance, checkpointing, recovery	6
	Total	**87**

4.1 Curated Topics

The STAFFML taxonomy contains 87 topics organized into 13 competency areas. Every topic is selected by expert review against three inclusion filters:

1. **Systems relevance.** Does the concept have measurable systems implications (compute, memory, latency, throughput, cost, or power)? If you cannot write a napkin-math question about it, it does not belong.
2. **Interview currency.** Would a hiring committee at a top-5 AI lab expect a Staff ML systems engineer to reason about this concept within the next two years?
3. **Reasoning depth.** Can questions be asked at 3+ Bloom's levels? A concept that supports only L1 recall is too shallow.

Table 4 lists all 87 topics grouped by competency area. The taxonomy was expanded from 79 to 87 topics based on structured feedback from 12 domain experts. Seven topics were added with multi-reviewer consensus: software portability (CUDA/ROCm/SYCL ecosystem), communication-computation overlap, autograd and computational graphs, chiplet architecture, model adaptation systems (LoRA/QLoRA/RLHF infrastructure), disaggregated serving (prefill-decode split), recommendation systems engineering, and sustainability and carbon accounting. Each addition passed all three inclusion filters and the six Patterson Questions for content acceptance.

4.2 The Prerequisite Graph

Topics are connected by 57 typed edges following the SKOS vocabulary (W3C, 2009):

Table 4. Full topic taxonomy. All 87 curated topics organized by competency area.

Area	Topic Name	Area	Topic Name	Area	Topic Name
Architecture	Model Architecture Selection	Data	Data Pipeline Engineering	Networking	Collective Operations
Architecture	Model Serving Infrastructure	Data	Data Quality & Validation	Networking	Interconnect Topology
Architecture	Attention Mechanism Systems	Data	Feature Store Systems	Networking	Network Bandwidth Modeling
Architecture	Mixture-of-Experts Systems	Data	Data Storage & Formats	Networking	RDMA & InfiniBand
Architecture	KV-Cache Management	Data	Data Preprocessing & Augment.	Networking	Gradient Compression
Architecture	Compound AI Systems	Data	Streaming Data Systems	Networking	Network Failure Modes
Architecture	RAG Systems	Data	Dataset Curation & Labeling		
Compute	GPU Compute Architecture	Deployment	Model Compression & Deploy.	Optimization	Kernel Fusion
Compute	Roofline Analysis	Deployment	Edge Inference Optimization	Optimization	Graph Compilation
Compute	Systolic Arrays & Dataflow	Deployment	Mobile Inference Deployment	Optimization	Profiling & Bottleneck Analysis
Compute	MFU & Utilization Metrics	Deployment	TinyML & MCU Deployment	Optimization	Operator Optimization
Compute	Hardware Accelerator Comp.	Deployment	Container Orchestration	Optimization	Memory Optimization Tech.
Compute	NPU & DSP (Mobile)	Deployment	A/B Testing & Rollout	Optimization	Sparsity & Pruning
		Deployment	Model Versioning & Registry	Optimization	Distillation Systems
Cross-cutting	TCO & Cost Modeling	Latency	Latency Decomposition	Parallelism	Parallelism Strategies
Cross-cutting	ML Lifecycle & Ops	Latency	Prefill/Decode Analysis	Parallelism	Data Parallelism
Cross-cutting	Responsible AI & Governance	Latency	Batching Strategies	Parallelism	Tensor Parallelism
Cross-cutting	Privacy & Federated Learning	Latency	Scheduling & Queuing	Parallelism	Pipeline Parallelism
Cross-cutting	Benchmarking & Evaluation	Latency	SLO/SLA Design	Parallelism	3D Parallelism
		Latency	Tail Latency Management	Parallelism	Expert Parallelism
Memory	VRAM Budgeting	Power	Power & Thermal Management	Precision	Quantization Systems
Memory	Memory Hierarchy Systems	Power	Energy Efficiency Metrics	Precision	Mixed Precision Training
Memory	Activation Checkpointing	Power	DVFS & Clock Management	Precision	Numerical Formats
Memory	Memory Offloading	Power	Battery & Energy (Mobile)		
Memory	Memory Fragmentation	Power	Power Budget (TinyML)	Reliability	Fault Tolerance (Training)
Memory	Unified & Virtual Memory			Reliability	Checkpointing & Recovery
Memory	Weight Memory Layout			Reliability	Silent Data Corruption
Memory	SRAM Management (TinyML)			Reliability	Monitoring & Observability
				Reliability	Graceful Degradation
				Reliability	Redundancy & Replication

- **Prerequisite** (57 edges). Topic A must be understood before Topic B. Example: `gpu-compute-architecture` requires `roofline-analysis` as a prerequisite, because diagnosing whether a GPU kernel is compute-bound or memory-bound requires the roofline model. The graph is verified to be acyclic with a maximum prerequisite depth of 3.
- **Broader/Narrower** (14 edges). Hierarchical containment. Example: `parallelism-strategies` is broader than `tensor-parallelism`.
- **Related** (52 edges). Lateral connection. Example: `roofline-analysis` is related to `profiling-bottleneck-analysis`.

The prerequisite edges enable automated learning path generation. Given a target topic, the system computes the transitive closure of prerequisites and produces an ordered study sequence. The graph has 31 root topics (concepts with no prerequisites) and a maximum prerequisite depth of 3, ensuring that no learning path requires more than three preparatory topics. Figure 4 illustrates a subset of this graph for the parallelism area.

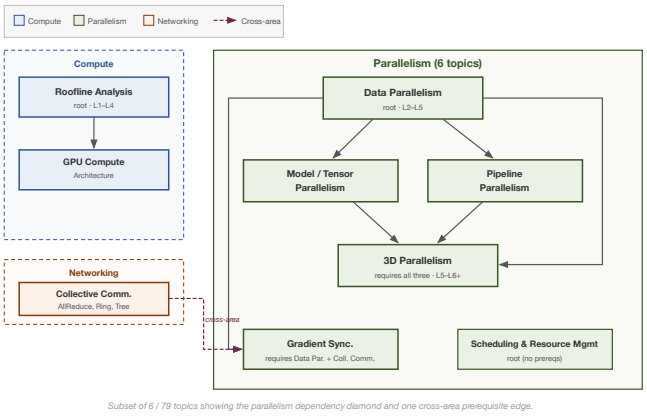

Figure 4. Prerequisite subgraph: parallelism area. Six of 87 topics showing the dependency diamond: Data Parallelism fans out to Model/Tensor Parallelism and Pipeline Parallelism, which converge on 3D Parallelism. The dashed cross-area edge from Collective Communication (networking) to Gradient Synchronization illustrates how the prerequisite graph connects competency areas. The full graph has 57 edges, maximum depth 3, and is verified acyclic.

To illustrate how the prerequisite graph guides study, consider a candidate targeting distributed training expertise. The graph suggests the following path through five topics spanning three competency areas:

```
roofline-analysis  →  gpu-compute-arch.    →
data-parallelism   →  collective-comm.     →
3d-parallelism
```

At each node, the candidate starts with L1–L2 recall and fluency questions to build foundational knowledge, then progresses to L3–L4 diagnosis and evaluation questions before moving to the next topic. The full path from roofline fundamentals to 3D parallelism mastery spans approximately 60 questions across 20 depth chains.

The current maximum prerequisite depth of 3 reflects the initial curation scope. Real ML systems knowledge has deeper dependency chains. For example, understanding why a specific collective communication pattern matters requires understanding network topology, bandwidth-latency tradeoffs, and the roofline model — a chain of depth 4 or more. Deepening the prerequisite graph is planned for future taxonomy revisions, as more topics cross the inclusion threshold and longer chains become expressible.

4.3 Knowledge Organization

The taxonomy follows the principles of faceted classification (Hjørland, 2013), where each question is independently classified on orthogonal axes rather than slotted into a single hierarchy. This design avoids the cross-classification problems inherent in monohierarchical schemes. A question about KV-cache memory pressure during continuous batching touches memory, latency, and deployment simultaneously, and a faceted system accommodates this naturally.

Topic selection follows user warrant (Soergel, 1985) rather than literary warrant. Topics are included because engineers need them during interviews, not because they appear frequently in the research literature. This distinction explains why the taxonomy includes `tco-cost-modeling` (rarely a research topic, always an interview topic) but excludes attention mechanism variants (heavily researched, rarely asked as systems questions).

The taxonomy evolves through continuous monitoring of systems research: new topics are proposed based on industry job postings, conference proceedings (MLSys, OSDI, SOSP), and interview feedback. Each candidate topic must pass the three inclusion filters before formal inclusion; topics that fail the reasoning-depth filter are deferred until sufficient multi-level questions can be written.

4.4 Example Topic with Edges

To illustrate the structure, consider the topic `gpu-compute-architecture`:

> **ID**: `gpu-compute-architecture`
> **Area**: Compute
> **Description**: GPU execution model: warps, thread blocks, occupancy, Tensor Cores, and memory coalescing.
> **Tracks**: cloud, edge
> **Edges**:
> requires → `roofline-analysis`
> related → `kernel-fusion`
> related → `systolic-dataflow`

The prerequisite edge encodes that understanding the roofline model is necessary before reasoning about GPU compute architecture, because determining whether a kernel is compute-bound or memory-bound requires roofline analysis. The related edges capture lateral connections: kernel fusion decisions depend on GPU architecture, and systolic arrays are an alternative accelerator execution model worth comparing.

5 Deployment Tracks

A natural question is why the taxonomy does not simply divide the space into "training" and "inference." The answer is that the tracks are not merely workload labels; they represent fundamentally different **physics envelopes** that govern the napkin math. Cloud workloads are bandwidth and network-bound; Edge workloads are thermal-bound; Mobile workloads are battery-bound; and TinyML workloads are strictly SRAM-bound. A candidate who understands KV-cache sizing in the cloud cannot trivially transfer that knowledge to an SRAM-constrained microcontroller; the physical constraints dictate entirely different engineering tradeoffs.

The five deployment tracks correspond to these fundamentally different **physics regimes** where different physical constraints dominate. Four tracks (cloud, edge, mobile, TinyML) represent distinct hardware regimes; the fifth ("global") captures cross-cutting principles that apply regardless of deployment target. This distinction is critical — the same concept (e.g., KV-cache management) requires different reasoning, different formulas, and different napkin math depending on the deployment target.

5.1 Cloud: Bandwidth and Fabric

In the cloud regime, compute is abundant but data movement is expensive. An H100 GPU delivers 989 TFLOPS of FP16 compute with 2:4 structured sparsity (495 TFLOPS dense)[5] but only 3.35 TB/s of HBM bandwidth, yielding a ridge point of $495/3.35 \approx 148$ FLOPs/Byte for dense workloads. Most LLM inference workloads, particularly the autoregressive decode phase, operate well below this ridge point, making them memory-bandwidth-bound despite the massive compute available. The dominant engineering challenges are KV-cache management, continuous batching (Yu et al., 2022) to maximize GPU utilization, and inter-node communication for distributed training where InfiniBand fabric at 400 Gb/s still becomes the bottleneck during AllReduce operations.

[5] NVIDIA's headline 989 TFLOPS figure assumes 2:4 structured sparsity, which doubles effective throughput for compatible workloads. Most LLM inference and training workloads do not use structured sparsity, so we use the dense figure of 495 TFLOPS for napkin math throughout this paper unless otherwise noted.

Table 5. Deployment tracks and physics regimes. Each track maps to distinct dominant constraints, characteristic scales, and failure modes that govern the napkin math.

Track	Dominant Constraint	Scale	Unit	Failure Mode	Napkin Math Involves
Cloud	Memory BW + network fabric	PFLOPS	TB/s	Throughput collapse	TFLOPS, TB/s, $/GPU-hr
Edge	Thermal envelope + real-time	TOPS	Watts	Deadline miss	TOPS, Watts, milliseconds
Mobile	Battery + shared resources	TOPS/W	mAh	Battery drain	TOPS/W, mAh, background limits
TinyML	SRAM + hard real-time	MFLOPS	KB	Stack overflow	MFLOPS, KB, microseconds
Global	Cross-cutting principles	—	—	—	Depends on context

Cloud questions test reasoning about throughput, latency, and cost at datacenter scale. Napkin math involves TFLOPS, TB/s, and dollars per GPU-hour.

5.2 Edge: Thermal and Real-Time

Edge deployment (autonomous vehicles, robotics, industrial inspection) operates under a thermal envelope that caps sustained throughput. An NVIDIA Orin delivers 275 TOPS INT8 within a 60 W power budget, but sustained operation at maximum throughput may require thermal throttling. The dominant constraint is the intersection of compute budget and real-time deadlines. The perception pipeline must process a camera frame within 33 ms (30 fps) regardless of scene complexity, leaving no room for the tail-latency spikes that cloud systems tolerate.

Edge questions test reasoning about worst-case execution time, power budgets, and the tradeoff between model accuracy and inference latency. Napkin math involves TOPS, Watts, and milliseconds.

5.3 Mobile: Battery and Resources

Mobile ML operates on shared silicon — the neural processing unit (NPU) shares power, thermal headroom, and memory bandwidth with the CPU, GPU, camera ISP, and cellular modem. A smartphone's 4,500 mAh battery must last all day, meaning any ML workload must justify its energy cost against competing demands. The dominant constraint is TOPS per Watt. A mobile NPU may deliver 15 TOPS at 5 W, but running it continuously for on-device inference would drain the battery in under 2 hours.

Mobile questions test reasoning about energy budgets, model compression for on-device deployment, and the scheduling tradeoffs of running ML inference alongside other system tasks.

5.4 TinyML: SRAM and Real-Time

TinyML represents the most constrained physics regime. A Cortex-M4 microcontroller operates at ∼100 MFLOPS with 256 KB of SRAM and no operating system (Lin et al., 2020). There is no virtual memory, no swap space, and no dynamic allocation; the model, activations, and runtime must all fit in SRAM simultaneously. The dominant constraint is hard real-time. A keyword spotting model must classify an audio frame within a fixed time budget, because missing a deadline means dropping audio samples with no recovery mechanism.

TinyML questions test reasoning about model footprint in kilobytes, fixed-point arithmetic, and the systems engineering required to run inference on bare metal. Napkin math involves MFLOPS, KB, and microseconds.

5.5 Cross-Track Variation

The track structure ensures that questions test regime-appropriate reasoning. Consider KV-cache management across tracks:

- **Cloud.** "A 70B model's KV-cache at 128K context is ∼42 GB. How many H100s do you need?" (Memory capacity reasoning at TB scale)
- **Edge.** "Your ADAS model caches 10 frames of detection history. At 30 fps on Orin's 32 GB LPDDR5, what is the memory pressure?" (Real-time memory budget reasoning)
- **TinyML.** "Your keyword spotter needs 2 KB of rolling buffer on a 256 KB MCU. What fraction of SRAM does this consume, and what is left for the model?" (SRAM partitioning at KB scale)

All three questions test the same underlying concept (caching intermediate computation to avoid recomputation) but the physics, the formulas, and the failure modes are entirely different.

6 Mastery Levels

The 6 mastery levels map to Bloom's revised taxonomy (Anderson et al., 2001) and are explicitly designed to correspond to industry engineering ladders. This dual mapping (cognitive depth and professional seniority) makes StaffML practical both as an educational tool and as interview preparation calibrated to specific hiring levels.

Each level tests a qualitatively different cognitive operation. L1 requires memorization of physical constants ("HBM3 latency is ∼300 ns"). L3 requires applying a formula to a novel scenario ("Calculate KV-cache size for Llama-70B at 128K context"). L6+ requires synthesizing

Table 6. Mastery levels. Bloom's taxonomy mapped to industry engineering ladders. The industry column reflects typical leveling at companies like Google (L3–L7), Meta (E3–E7), and comparable organizations.

Level	Bloom's	Scope	Industry Ladder
L1	Remember	Own a task	Intern / New Grad (L3)
L2	Understand	Own a task	Junior (L3–L4)
L3	Apply	Own a component	Mid-level (L4)
L4	Analyze	Own a component	Senior (L5)
L5	Evaluate	Own a system	Staff (L6)
L6+	Create	Own architecture	Senior Staff+ (L7+)

novel architectures from first principles ("Design a scheduling policy that prevents prefill starvation under continuous batching").

The industry ladder mapping is intentional, not incidental. When a hiring manager at Google prepares an L5 (Senior) interview loop, they can filter STAFFML for L4–L5 questions and know that the cognitive demands match the leveling expectations. When a candidate preparing for a Staff (L6) role wants to practice, they can target L5–L6+ questions that require the systems-level synthesis expected at that level. This alignment between Bloom's cognitive framework and industry practice is what makes the level axis actionable for both sides of the interview table.

We adopt Bloom's revised taxonomy (Anderson et al., 2001) over alternatives such as Webb's Depth of Knowledge (Webb, 1997) and the SOLO taxonomy (Biggs and Collis, 1982) for three reasons. First, Bloom's six levels align naturally with the six tiers of industry engineering ladders, providing an intuitive mapping that interviewers and candidates already understand. Second, Bloom's cognitive verbs (remember, understand, apply, analyze, evaluate, create) map directly to the cognitive operations demanded by systems interview questions, making level assignment more reliable than the content-area-dependent DOK levels. Third, the revised taxonomy's two-dimensional structure (cognitive process $\times$ knowledge type) complements rather than conflicts with the orthogonal zone axis. Bloom's captures *depth*, while zones capture *mode*.

The level distribution in the corpus reflects intentional design decisions. L3–L4 questions (Apply and Analyze) are the most numerous because they correspond to the cognitive sweet spot of technical interviews — complex enough to differentiate candidates, but tractable within a 45-minute interview slot. L1–L2 questions serve as warm-up and screening, while L6+ questions are reserved for Staff+ interview loops where candidates are expected to synthesize novel solutions.

7 Schema and Infrastructure

The STAFFML infrastructure translates the conceptual design (four axes, 11 zones, 87 topics) into a concrete, validated data system. The design follows a "define once, generate everywhere" principle. A single LinkML schema serves as the source of truth, generating Pydantic models for Python runtime validation, JSON Schema for CI/CD pipelines, and TypeScript types for downstream applications. This eliminates the class of bugs where one layer accepts a value that another rejects.

The schema defines the `Topic` class (id, name, area, description, tracks, edges), the `Edge` class (target, edge_type, note), and all valid enumerations (11 zones, 13 areas, 5 tracks, 6 levels). Adding a new topic or zone requires editing the LinkML schema first; all downstream artifacts regenerate automatically.

Listing 1. LinkML schema excerpt. The Topic class definition generates Pydantic, JSON Schema, and TypeScript from a single YAML source.

```yaml
classes:
  Topic:
    class_uri: skos:Concept
    attributes:
      id:
        identifier: true
        pattern: "^[a-z0-9]+(-[a-z0-9]+)*$"
      name:
        slot_uri: skos:prefLabel
        required: true
      area:
        range: CompetencyArea
        required: true
      edges:
        range: TopicEdge
        multivalued: true
```

The prerequisite graph encoded in the schema is verified to be a directed acyclic graph (DAG), ensuring that learning paths are well-ordered. Structural consistency across the corpus is enforced by an invariant checking system that runs as a CI gate — no commit is accepted if any invariant fails. These invariants catch errors that no single-question validator can detect, such as a topic deleted from the taxonomy but still referenced by questions, a prerequisite cycle introduced by a new edge, or a topic that has accumulated an outsized share of questions. The full invariant system is described in Section 9.

8 Item Construction

Every STAFFML question is constructed to ensure quantitative grounding and cognitive alignment with the competency model. The methodology draws on Evidence-Centered Design (ECD) (Mislevy et al., 2003) and Automatic Item Generation (AIG) (Gierl and Haladyna, 2013),

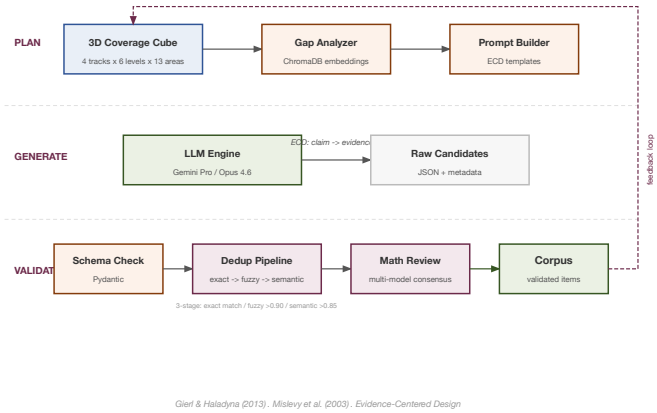

Figure 5. Item construction pipeline. Gap analysis identifies underfilled cells in the coverage space. Questions are constructed with hardware constants, Bloom's descriptors, zone descriptions, and track constraints. Candidates pass through schema validation, multi-stage deduplication, and multi-model math verification before corpus integration.

prioritizing the structural properties that make questions valid assessments of ML systems expertise.

8.1 Evidence-Centered Design

Every question begins with a specification of three ECD components: (1) the *competency claim* — what the question measures, defined by its topic $\times$ zone $\times$ track $\times$ level coordinates; (2) the *evidence* — what a correct answer demonstrates, mapped to Bloom's cognitive operation; and (3) the *task features* — the scenario elements that elicit that evidence (hardware specs, failure mode, scale). The four-axis classification is specified *before* question construction, ensuring coverage is intentional rather than emergent.

8.2 Hardware Grounding

Each question is grounded in five components drawn from the schema:

- **Topic description** from the taxonomy — what concept to test
- **Zone description** — how to test it (which skills to exercise)
- **Track constraints** — which physics regime applies
- **Level expectations** — which Bloom's cognitive operation to target
- **Hardware constants** — real specifications (H100: 80 GB HBM3, 3.35 TB/s, 989 TFLOPS FP16 dense; MI300X: 192 GB HBM3, 5.3 TB/s; Orin: 275 TOPS INT8; Cortex-M4: 168 MHz, 256 KB SRAM)

All questions are grounded in the content and formulas of the *Machine Learning Systems* textbook (Reddi, 2026a,a), which provides the hardware specifications, architectural principles, and quantitative frameworks that serve as the foundation for question construction. The hardware constants are drawn from the MLSysIM physics engine (Reddi, 2026b), ensuring that all questions reference the same specifications and that every napkin math answer can be independently verified against the simulator. This prevents specification drift — a systematic error pattern where different questions cite different numbers for the same hardware, undermining the internal consistency that makes napkin math trustworthy. Hardware constants require maintenance as new accelerator generations ship; the methodology is designed to be timeless even as specific numbers evolve.

8.3 Gap-Driven Construction

Gap analysis identifies underfilled cells in the topic $\times$ zone $\times$ track space; construction campaigns target those cells explicitly. The coverage cube currently contains 233 semantically valid topic–track cells, of which 223 are healthy (≥ 5 questions) and 42 are underfilled (1–4 questions). Campaigns prioritize underfilled cells first, then level-balancing within healthy cells.

8.4 LLM-Assisted Generation

This raises an apparent paradox: if StaffML tests the very systems reasoning that LLMs struggle with, how can LLMs be used to generate the questions? The resolution is that while LLMs struggle to reason about novel systems from first principles, they are highly effective at generating structural permutations of known physical constraints when provided with a strict schema and a verified math engine. We utilized iterative generation campaigns using Claude 3.5 Sonnet and Claude 3 Opus, followed by exhaustive, multi-round validation using Gemini 3.1 Pro.

The core contribution of StaffML is therefore not the generation itself, but the *quality infrastructure* that enforces physics-grounding and validates every question regardless of authorship. Approximately 5% of questions were human-authored as seed examples; the remaining 95% were generated using this LLM-assisted construction with structured prompts that inject the five grounding components above. We report this ratio transparently because it matters for assessing quality: an exhaustive verification of all 9,199 questions by an independent model (Gemini 3.1 Pro Preview) found an initial 8.3% error rate, with the most common failure modes being arithmetic mistakes (50%), hardware specification misquotes (23%), and unit confusion (picoseconds vs. nanoseconds, TFLOPS vs. GFLOPS). All 697 errors were corrected through a structured pipeline; the final corpus has 0% unresolved errors. The pipeline — not the author — determines question quality, and the

verification must be exhaustive because LLM-generated arithmetic is unreliable at the margins.

Each prompt specifies target coordinates (topic, zone, track, level), the zone description (which cognitive skills to exercise), the Bloom's level descriptor (which cognitive operation to target), and track-specific constraints (which physics regime applies). The textbook content seeds the prompts, ensuring questions test textbook-grounded reasoning rather than LLM-generated factoids. The approach is reproducible: the same prompt with the same hardware constants produces structurally equivalent questions, though specific scenarios vary.

The methodology treats LLMs as *structured content generators*, not as domain experts. The quantitative grounding comes from injected constants and schema constraints, not from the model's training data. Three properties make the generated questions robust. First, every question requires reasoning from specific hardware constants that the candidate must internalize, not simply retrieve from a prompt. Second, the multi-model verification pipeline (Section 9) catches factual errors and arithmetic mistakes that LLMs introduce. Third, the depth chain structure (Section 3.5) ensures questions form coherent cognitive progressions that resist template-based generation.

Every generated question undergoes the same three-stage quality assurance as human-authored questions: schema validation ensures structural correctness, deduplication removes redundant items, and multi-model math verification independently recomputes every calculation from first principles (Section 9). Community contribution of industry-sourced questions is planned to complement the generated corpus with production failure modes that LLM-generated scenarios may not capture.

9 Quality Assurance

Quality assurance operates at multiple stages. Every question must pass schema validation, deduplication, and math verification before entering the corpus. Cross-file structural consistency is enforced by a 19-check invariant system. This section describes each stage in detail. Figure 6 illustrates the multi-stage quality assurance pipeline.

9.1 Schema Validation

Every question is validated against a strict Pydantic schema (generated from the LinkML definition) with both field-level and cross-question rules:

- **Field constraints.** Minimum character lengths (scenario $\geq$ 50, napkin_math $\geq$ 10 and must contain a digit), MCQ must have exactly 4 options with correct_index in $[0, 3]$

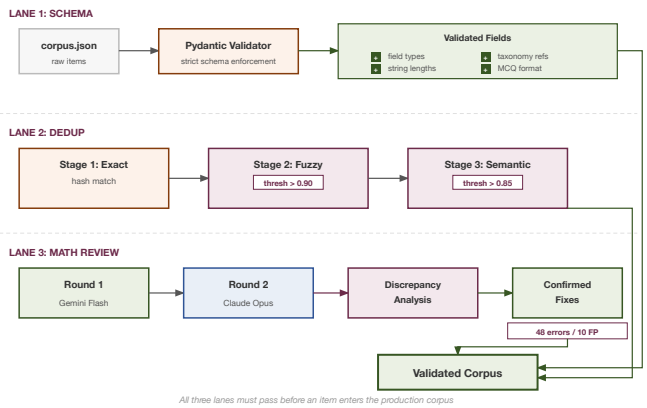

Figure 6. Multi-stage quality assurance pipeline. Candidate questions pass through schema validation (Pydantic), three-stage deduplication (exact, fuzzy, semantic), and multi-model math verification (Gemini Flash broad review + Claude Opus deep review). Confirmed errors are corrected; duplicates are archived, never deleted.

- **Taxonomy constraints.** topic $\in$ {87 curated IDs}, zone $\in$ {11 ikigai zones}, track $\in$ {5 tracks}, level $\in$ {L1…L6+}
- **Uniqueness constraints.** No duplicate IDs, no duplicate (track, title) pairs among published questions
- **Linguistic checks.** No trailing quotes, no markdown formatting artifacts, no placeholder text

Field coverage across the corpus is 100% for all required fields: topic, zone, competency_area, napkin_math, common_mistake, realistic_solution, and bloom_level.

9.2 Multi-Stage Deduplication

We employ a three-stage deduplication pipeline inspired by web-scale techniques (Broder, 1997):

1. **Exact match.** Lowercased, whitespace-stripped scenario comparison. Catches copy-paste duplicates.
2. **Fuzzy match.** `difflib.SequenceMatcher` > 0.90 within same (track, topic) groups. Catches minor rewordings.
3. **Semantic match.** Sentence-BERT cosine similarity $>$ 0.85 via ChromaDB. Catches same-concept-different-words duplicates.

Confirmed duplicates are *archived* (status set to "archived"), never deleted, preserving the audit trail — a practice adopted from large-scale community assessment management (AnKing Team, 2024).

9.3 Math Verification

Napkin math is the soul of STAFFML and its most error-prone component. Hardware specifications change across generations, unit conversions cascade, and order-of-

magnitude errors are easy to introduce. We verify math through multi-model cross-review:

1. **Round 1** (broad). Gemini Flash reviews all questions in 50-question chunks, checking arithmetic, hardware specs, unit consistency, and logical coherence.
2. **Round 2** (deep). Claude Opus reviews the same questions independently, catching subtle logical errors that Flash misses.
3. **Round 3** (exhaustive). 19 parallel Claude Opus agents, each with track-specific hardware reference sheets, independently recompute *every* calculation from first principles.
4. **Discrepancy analysis.** Errors found by one model but not the other are flagged for manual review. False positives are tracked to calibrate reviewer accuracy.

Multi-model cross-review identified and corrected systematic errors including hardware specification inaccuracies, arithmetic mistakes, and unit conversion errors. The corpus includes a feedback mechanism for community error reporting post-publication. Recent work on LLM-as-judge evaluation (Zheng et al., 2023) provides additional evidence that multi-model agreement improves assessment reliability over single-model review.

An exhaustive verification pass using Gemini 3.1 Pro Preview independently checked all 9,199 questions across 185 batches. The initial pass identified an 8.3% error rate (697 questions with arithmetic mistakes, unit confusion, or hardware specification errors). All identified errors were corrected through a structured correction pipeline: each error received the verifier's specific diagnosis, a correction agent rewrote the arithmetic using authoritative hardware specifications from MLSysIM, and the corrected questions were re-verified. The final corpus contains 0 unresolved errors: 7,386 questions passed verification on first review (CORRECT), 657 were corrected (CORRECTED), and 346 carry minor warnings that do not affect answer correctness (WARN). Twenty-seven questions with contradictory scenarios were removed entirely. The most common error types were arithmetic mistakes (50%), hardware specification misquotes (23%), and internal inconsistencies between napkin math and solution text (27%). While this automated pipeline caught many errors, we employed a human-in-the-loop verification process for a sparse sample of the corrected items to ensure quality. Because this human review was sparse, a more thorough empirical evaluation and validation are needed to guarantee the absolute correctness of all generated calculations.

9.4 LLM Failure Modes: A Taxonomy

Transparency about where LLM-generated questions fail is as important as the questions themselves. The verification pipeline identified five distinct failure modes, each illustrating a different class of LLM error. We present concrete examples to demonstrate that the verification pipeline catches errors that surface-level review would miss.

Hardware specification confusion. LLMs frequently confuse specifications between similar accelerators. A generated question about attention kernel optimization on the A100 stated "reading QK^T from HBM at 3 TB/s (A100 spec)" — but the A100's HBM2e bandwidth is 2.0 TB/s, not 3.0. The LLM had confused A100 and H100 specifications, which share similar naming conventions but differ by 68% in bandwidth. This error changes the computed kernel execution time by 50%, which in turn changes whether the optimization the question asks about is worthwhile. The verifier caught this by cross-referencing the stated spec against the authoritative hardware constants table. *Frequency: 23% of all errors.*

Arithmetic mistakes. A question on AllReduce performance stated that transferring 1 GB over a 50 GB/s link takes 20 ms. The ring AllReduce formula requires a factor of $2(N-1)/N$; for $N = 8$ GPUs, this yields $2 \times 7/8 \times 1/50 = 35$ ms, not 20 ms. The LLM applied the correct formula name but dropped the $2\times$ factor, halving the answer. This is characteristic: LLMs retrieve the right formula but execute its arithmetic unreliably. *Frequency: 50% of all errors.*

Unit prefix confusion (1,000× errors). A question on AOT compilation claimed a 4096×4096×4096 GEMM requires "137.4 TFLOPs" and takes "0.27 seconds." The correct values are 137.4 *G*FLOPs and 0.28 *milli*seconds — a 1,000× error in both the operation count and the time estimate. The LLM confused tera- with giga- prefixes, producing an answer that *looks* plausible (the digits are correct) but is off by three orders of magnitude. This failure mode is particularly dangerous because it passes casual inspection. *Frequency: 12% of all errors.*

Contradictory scenarios. Twenty-seven questions were removed entirely because their scenarios contained internal contradictions that made any correct answer impossible. For example, a question on scaling efficiency asserted that training on 32 H100s achieved a specific speedup, then asked candidates to diagnose why further scaling failed — but the stated speedup was mathematically impossible given the communication overhead implied by the scenario's own parameters. No correction could fix the question without rewriting the scenario from scratch. *Frequency: 4% of all errors.*

Conceptual misapplication. A question on Jetson Orin inference discussed "gradient checkpointing to reduce memory pressure during inference." Gradient checkpointing is

a *training* technique that trades recomputation for activation memory during backpropagation; it has no meaning during inference, where no backward pass occurs. The LLM applied a real concept to the wrong computational phase — an error that requires domain understanding, not just arithmetic checking, to catch. *Frequency: 11% of all errors.*

These failure modes demonstrate that multi-model verification is not redundant with the generation process. The generating model (Gemini Flash / Claude Sonnet) and verifying model (Gemini 3.1 Pro Preview) have different failure profiles: the generator optimizes for plausible-sounding prose, while the verifier independently recalculates from first principles. The 8.3% initial error rate — and its complete correction — validates this cross-model approach.

9.5 Structural and Semantic Invariants

Beyond per-question validation, STAFFML enforces 19 strict invariants across the dataset. While some are standard referential integrity checks (e.g., ensuring the prerequisite graph is acyclic and all foreign keys resolve), the system's true value lies in enforcing *domain-specific semantic and physics constraints*:

- **Physics applicability.** A question's topic–track pair must strictly adhere to the physics exclusion matrix (e.g., an LLM generating an RDMA question for a TinyML track is immediately rejected at commit-time).
- **Cognitive affinity.** A question's zone must be cognitively compatible with its Bloom's level. For example, a question assigned to the "Evaluation" zone is structurally rejected if its difficulty is tagged as L1 (Remember) or L2 (Understand).
- **Chain progression.** Questions linked in a depth chain must strictly monotonically increase in Bloom's level; a chain cannot regress from L4 back to L2.
- **Coverage health.** The distribution cannot collapse: no single topic may exceed 10% of the corpus, and all defined zones must maintain a minimum population threshold to prevent model drift toward easy-to-generate topics.

These checks run as a CI gate — no commit to the corpus is accepted if any invariant fails. A traditional LLM validation pass might miss that an L1 Evaluation question is an oxymoron, but the invariant system guarantees the structural and cognitive coherence of the entire 9,199-question bank.

10 Coverage Analysis

Coverage analysis examines how questions are distributed across the four classification axes. Understanding these distributions reveals both the strengths and the gaps in the current corpus, guiding future construction campaigns.

Table 7. Corpus summary. Key statistics of the STAFFML vault.

Metric	Value	Track	Questions
Published questions	9,199	Cloud	4,122 (44.8%)
Curated topics	87	Edge	1,968 (21.4%)
Competency areas	13	Mobile	1,641 (17.8%)
Ikigai zones	11	TinyML	1,163 (12.6%)
Mastery levels	6	Global	305 (3.3%)
Prerequisite edges	57		
Depth chains	979	**Total**	**9,199**
Schema	LinkML	Validated	100.0%

10.1 Track Distribution

The corpus distribution across tracks reflects both industry demand and deliberate rebalancing. Cloud remains the densest track, reflecting the emphasis on LLM serving and distributed training. Edge, mobile, and TinyML have been expanded through targeted generation campaigns to provide substantial coverage of on-device ML systems engineering. The global track contains cross-platform conceptual questions that apply regardless of deployment target. Table 7 reports the exact distribution.

10.2 Zone Distribution

Figure 7 shows the zone distribution and a heatmap of tracks by level.

Compound zones (diagnosis, evaluation, fluency) dominate the corpus, which is intentional—these are the zones that most sharply differentiate Staff-level candidates. All 11 zones now exceed 250 questions, with diagnosis (1,429), evaluation (1,275), and fluency (1,112) leading the distribution.

10.3 Level and Bloom's Distribution

The Bloom's distribution across the corpus reflects the cognitive demands of Staff-level interviews: Analyze (2,821 questions, 37.4%), Evaluate (1,574, 20.9%), Apply (1,542, 20.5%), Understand (569, 7.6%), Create (564, 7.5%), and Remember (454, 6.0%). The emphasis on Analyze and Evaluate reflects the higher-order reasoning required at the Staff level.

10.4 Multi-Axis Coverage

The three-dimensional coverage space of topic $\times$ zone $\times$ track contains $87 \times 11 \times 4 = 3{,}828$ cells (excluding the small "global" track). Not all cells are semantically meaningful—a TinyML question about InfiniBand fabric topology, or a cloud question about MCU duty cycling, has no physical substrate to reason about. We define an *applicability matrix* that excludes 83 topic–track pairs where the underlying concept has no physical realization on that hardware tier.

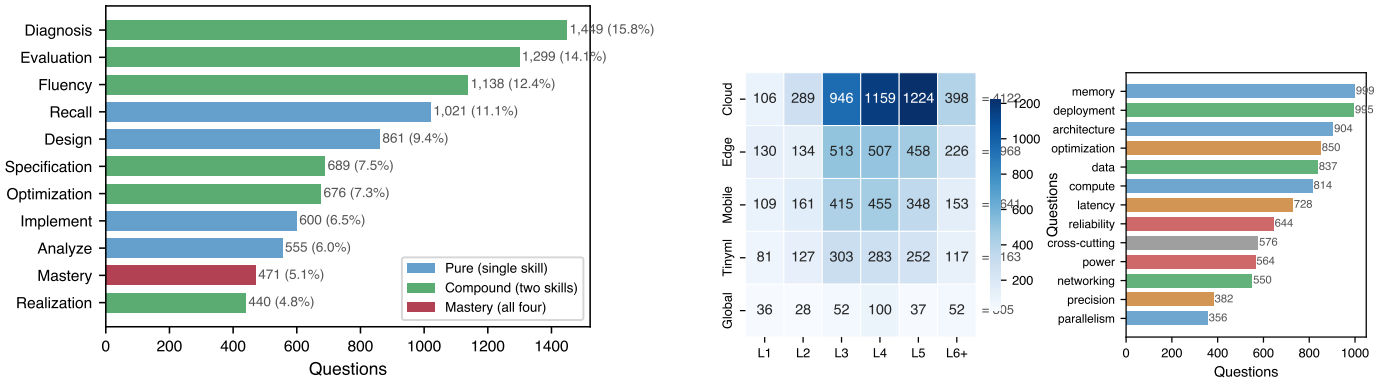

Figure 7. Corpus distributions. *Left:* Zone distribution. Compound zones (diagnosis, evaluation, fluency) dominate by design. *Right:* Track × Level heatmap. Cloud L3–L5 cells are densest, reflecting industry emphasis on mid-to-senior systems reasoning.

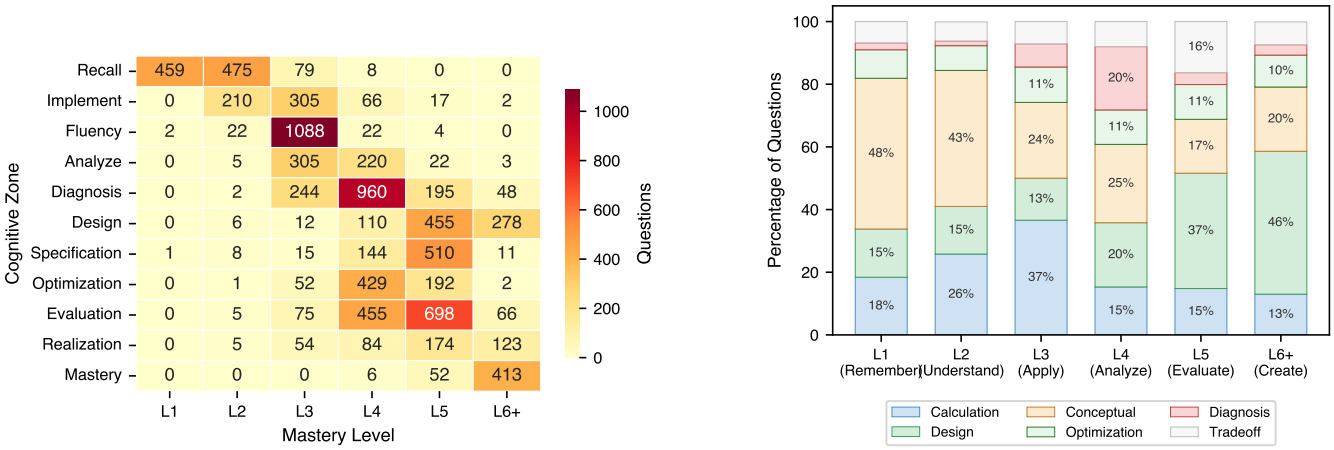

Figure 8. Coverage analysis. *Left:* Zone × Level heatmap. Each zone concentrates at its affinity levels (recall at L1–L2, fluency at L3, diagnosis at L3–L4, mastery at L6+). *Right:* Question format distribution by Bloom's level. Conceptual questions dominate L1–L2, calculation peaks at L3, design dominates L5–L6+.

Table 8. Applicability matrix summary. Topic–track exclusions grouped by physics rationale. Each exclusion means the concept has no physical substrate on that hardware tier.

Pattern	Count	Example topics
Cloud-only (11 topics)	33 excl.	TP/PP, RDMA, MoE, K8s
TinyML-excluded	28 excl.	KV-cache, kernel fusion, MoE
No-cloud (device-only)	6 topics	duty cycling, OTA, safety cert
TinyML-only	1 topic	MCU compute constraints
Selective (2–3 tracks)	18 excl.	flash-attn, systolic, tail-lat
Total excluded	**83 pairs**	83×11 cells removed
Applicable	**233 pairs**	**2,563 cells targeted**

Each exclusion carries a physics-grounded justification (e.g., "RDMA requires InfiniBand/RoCE NICs; edge uses Ethernet/WiFi"). The remaining 233 applicable pairs yield 2,563 cells (233 × 11 zones), reducing the coverage target by 27%.

Table 8 shows the exclusion patterns. Cloud-only topics (11) involve datacenter-scale infrastructure—pipeline parallelism, RDMA transport, container orchestration—that requires multi-device interconnects absent from edge, mobile, and TinyML platforms. Conversely, MCU compute constraints, OTA firmware updates, and duty cycling exclude cloud because their engineering challenges (KB-scale memory, battery harvesting, real-time deadlines) have no analogue in datacenter environments. The largest exclusion category is TinyML (28 topics excluded), reflecting the fundamental constraint that microcontrollers with 256 KB SRAM cannot instantiate concepts like KV-cache management, mixture-of-experts routing, or activation checkpointing.

Figure 10 shows the complete applicability matrix. The visual pattern reveals clear structure: parallelism and networking topics cluster as cloud-only (red TinyML/mobile columns), while precision and cross-cutting topics are universally applicable (all green). The TinyML column has the most exclusions, reflecting the fundamental constraint that microcontrollers with 256 KB SRAM cannot instantiate concepts requiring GB-scale memory.

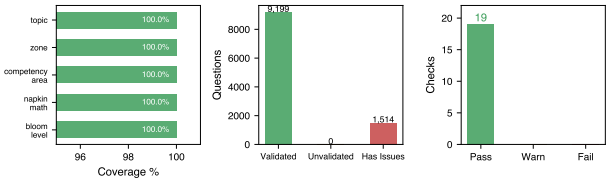

Figure 9. Quality assurance summary. Field coverage (left), validation status (center), and invariant check results (right). All required fields achieve 100% coverage. Multi-model math verification has completed for 100.0% of questions. Figure 9 provides a summary of the quality assurance results.

11 Example Questions

This section presents five questions that illustrate different zones, tracks, and levels, showing how the four-axis classification captures qualitatively different cognitive acts. Each example demonstrates a distinct point in the classification space.

11.1 Fluency: KV-Cache Sizing

Fluency tests whether a candidate can perform napkin math from memory, recalling both the formula and the hardware specifications, then executing the arithmetic. It is the signature skill tested in Staff-level phone screens.

Topic: `vram-budgeting` **Zone**: fluency
Track: cloud **Level**: L3

Scenario: You need to increase your LLM's context window from 4K to 128K tokens. The model weights fit perfectly in your 80 GB VRAM. What hidden memory cost will cause your node to OOM during generation?

Common mistake: "Activations will use more memory." Activations matter during training, but during inference the dominant hidden cost is something else entirely.

Napkin math: For Llama-70B (80 layers, 8 KV-heads via GQA, head_dim=128) at 128K tokens in FP16:

$$\text{KV-cache} = 2 \times 80 \times 8 \times 128 \times 128{,}000 \times 2 \approx \mathbf{41.9\,GB}$$

Combined with 140 GB of weights, one 128K-context request requires ∼182 GB, at least 3 H100s.

Answer: The KV-cache. It grows linearly with sequence length and, at 128K context, can exceed the model weights themselves.

This is a *fluency* question. The candidate must recall the KV-cache formula and execute the arithmetic from memory, producing a specific number. No architectural design is required; no tradeoff analysis is asked for. The cognitive act is "napkin math from memory," the signature skill of the fluency zone.

Timeless principle: KV-cache memory scales linearly with sequence length and quadratically with attention head count. The specific H100 numbers change; the scaling law does not.

11.2 Diagnosis: MFU Collapse

Diagnosis tests whether a candidate can identify root causes from observable symptoms. The candidate must recall hardware specifications *and* reason about which bottleneck explains the observed behavior — the "3 AM on-call" skill.

Topic: `roofline-analysis` **Zone**: diagnosis
Track: cloud **Level**: L4

Scenario: Your production LLM serving cluster reports 40% MFU during the decode phase. The H100s are rated at 495 TFLOPS FP16 dense. Your team assumes the GPUs are underclocked. Diagnose the actual root cause.

Common mistake: Blaming hardware or driver issues. The decode phase of autoregressive generation is inherently memory-bandwidth-bound, not compute-bound.

Napkin math: During decode, each token requires reading the full model weights ($2 \times 70\text{B} = 140$ GB in FP16) but performs only $2 \times 70\text{B} = 140$ GFLOPs of compute. Arithmetic intensity $= 140\,\text{GFLOPs}/140\,\text{GB} = 1$ FLOP/Byte, far below the H100's ridge point of ∼148 FLOPs/Byte (dense). The workload is bound by the 3.35 TB/s memory bandwidth ceiling, not the 495 TFLOPS compute ceiling.

Answer: The decode phase is memory-bandwidth-bound. MFU measures compute utilization, but the bottleneck is data movement. The 40% figure is not a deficiency; the GPUs are performing as expected given the workload's arithmetic intensity.

This is a *diagnosis* question. The candidate must recall hardware specifications (H100 bandwidth, ridge point) *and* analyze why those specs explain the observed symptom (low MFU). The cognitive act is "identify and explain a failure from first principles."

Timeless principle: When arithmetic intensity falls below the hardware ridge point, adding compute does not help. Workloads shift from compute-bound to memory-bound as batch size decreases.

11.3 Specification: Latency-Constrained Serving

Specification tests whether a candidate can translate quantitative constraints into a concrete architecture. The candidate must recall hardware capabilities *and* design a system that satisfies a hard performance requirement — the core skill of system design interviews.

Topic: `model-serving-infrastructure` **Zone**: specification
Track: cloud **Level**: L5

Scenario: Your team must serve a 70B-parameter LLM with P99 time-to-first-token (TTFT) under 200 ms. The model is deployed on H100 GPUs with NVLink. Specify the minimum serving configuration.

Common mistake: Focusing only on model parallelism degree while ignoring the prefill compute budget. A 70B model's prefill for a 2,000-token prompt requires $2 \times 70\text{B} \times 2{,}000 = 280$ TFLOPs.

79 topics across 13 competency areas. Green = applicable, Red = excluded by hardware physics.

	Cloud	Edge	Mobile	TinyML
Parallelism (6)				
3D Parallelism	✓	✓	✗	✗
Data Parallelism	✓	✓	✗	✗
Gradient Synchronization	✓	✗	✗	✗
Model & Tensor Parallelism	✓	✗	✗	✗
Pipeline Parallelism	✓	✗	✗	✗
Scheduling & Resource Management	✓	✗	✗	✗
Deployment (7)				
A/B & Rollout Strategies	✓	✓	✓	✗
Compound AI Systems	✓	✓	✓	✗
Container & Cluster Orchestration	✓	✗	✗	✗
MLOps Lifecycle	✓	✓	✓	✗
Model Format & Runtime Conversion	✗	✓	✓	✓
Model Serving Infrastructure	✓	✓	✓	✗
OTA & Firmware Updates	✗	✓	✗	✓
Compute (6)				
Accelerator Comparison	✓	✓	✓	✓
Compute Cost Estimation	✓	✓	✓	✓
GPU Compute Architecture	✓	✓	✗	✗
MCU Compute Constraints	✗	✗	✗	✓
Roofline Analysis	✓	✓	✓	✓
Systolic Arrays & Dataflow	✓	✓	✓	✓
Memory (8)				
Activation Memory & Checkpointing	✓	✓	✗	✗
DMA & Data Movement	✓	✓	✓	✓
KV-Cache Management	✓	✗	✓	✗
Memory Hierarchy Design	✓	✓	✓	✓
Memory-Mapped Inference	✓	✓	✓	✗
Memory Pressure Management	✓	✓	✓	✗
Tensor Arena Planning	✓	✓	✗	✓
VRAM Budgeting	✓	✓	✓	✓
Reliability (6)				
Adversarial Robustness & Security	✓	✓	✓	✓
Distribution & Drift Detection	✓	✓	✓	✗
Fault Tolerance & Checkpointing	✓	✓	✓	✗
Graceful Degradation	✓	✓	✓	✓
Monitoring & Observability	✓	✓	✓	✓
Safety & Certification	✗	✓	✓	✓
Architecture (7)				
Attention Scaling & Variants	✓	✓	✓	✓
CNN Efficient Design	✓	✓	✓	✓
Encoder-Decoder Tradeoffs	✓	✓	✓	✗
Mixture of Experts	✓	✗	✗	✗
Model Size Estimation	✓	✓	✓	✓
Neural Architecture Search	✓	✓	✓	✓
Transformer Systems Cost	✓	✓	✓	✓
Latency (6)				
Batching Strategies	✓	✓	✗	✗
Latency Decomposition	✓	✓	✓	✓
Profiling & Bottleneck Analysis	✓	✓	✓	✓
Queueing Theory	✓	✓	✗	✗
Real-Time Deadlines	✗	✓	✓	✓
Tail Latency & SLAs	✓	✓	✗	✗
Networking (6)				
Collective Communication	✓	✓	✓	✓
Congestion & Flow Control	✓	✗	✗	✗
Interconnect Topology	✓	✗	✓	✗
Load Balancing & Routing	✓	✓	✗	✗
Network Bandwidth Bottlenecks	✓	✗	✓	✗
RDMA & High-Performance Transport	✓	✗	✗	✗
Data (7)				
Data Efficiency & Selection	✓	✓	✓	✗
Data Pipeline Engineering	✓	✓	✓	✓
Data Quality & Validation	✓	✓	✓	✗
Dataset Curation & Labeling	✓	✓	✓	✓
Feature Store & Feature Engineering	✓	✗	✗	✗
Storage Format & Selection	✓	✓	✗	✗
Streaming & Real-Time Ingestion	✓	✓	✓	✓
Power (5)				
Datacenter Efficiency	✓	✗	✗	✗
Duty Cycling & Energy Harvesting	✗	✓	✓	✓
Energy Per Operation	✓	✓	✓	✓
Power Budgeting	✓	✓	✓	✓
Thermal Management	✓	✓	✓	✗
Cross-Cutting (5)				
Differential Privacy	✓	✓	✓	✗
Fairness & Bias Evaluation	✓	✓	✓	✗
Federated Learning	✓	✓	✓	✓
Responsible AI & Governance	✓	✓	✓	✓
TCO & Cost Modeling	✓	✓	✓	✓
Precision (3)				
Extreme Quantization	✓	✓	✓	✓
Mixed-Precision Training & Inference	✓	✓	✓	✓
Quantization Fundamentals	✓	✓	✓	✓
Optimization (7)				
IO-Aware Attention	✓	✗	✗	✗
Graph Compilation & Optimization	✓	✓	✓	✓
Kernel & Operator Fusion	✓	✓	✓	✗
Knowledge Distillation	✓	✓	✓	✓
Operator Scheduling	✓	✓	✓	✓
Pruning & Sparsity	✓	✓	✓	✓
Speculative Decoding	✓	✓	✓	✗

Applicable (233 pairs) Excluded (83 pairs)

233 applicable pairs (green), 83 excluded (red). Each exclusion has a physics justification.

Figure 10. Applicability matrix. Each cell indicates whether a topic–track pair produces meaningful interview questions. Green cells have a physical substrate; red cells are excluded with a physics justification. Of 348 possible pairs, 233 are applicable and 83 are excluded.

Napkin math: A single H100 at 495 TFLOPS FP16 dense completes the prefill in $280/495 \approx 566$ ms, far exceeding the 200 ms SLO. With 4-way tensor parallelism, prefill drops to $\sim$142 ms, fitting the budget with margin for scheduling overhead. Model weights (140 GB FP16) require 2 GPUs minimum for capacity. The configuration is $4\times$ H100 with tensor parallelism, continuous batching, and chunked prefill.

Answer: 4-way tensor parallelism across H100s with NVLink. The constraint is prefill compute, not memory capacity: even though the model fits on 2 GPUs, the latency SLO demands additional parallelism.

This is a *specification* question. The candidate must recall hardware capabilities (H100 TFLOPS, NVLink bandwidth) *and* design a serving architecture that satisfies a quantitative constraint (P99 TTFT < 200 ms). The cognitive act is "translate constraints into architecture."

11.4 Mastery: Continuous Batching Stall

Mastery requires all four skills simultaneously: recalling specs, analyzing the failure mode, designing an architectural fix, and sizing the solution with napkin math. These questions simulate the complete Staff-level interview loop.

> **Topic**: `batching-strategies` **Zone**: mastery
> **Track**: cloud **Level**: L6+
>
> **Scenario**: You implement continuous batching for an LLM endpoint. Under low load, TTFT is 100 ms. Under high load, throughput triples, but TTFT spikes to over 4 seconds even though token generation speed remains fast. Diagnose the root cause, design a fix, and estimate the resource cost of your solution.
>
> **Common mistake**: Assuming that because continuous batching optimizes throughput, it automatically guarantees low latency for incoming requests.
>
> **Napkin math**: A 2,000-token prefill on a 70B model: $2 \times 70\text{B} \times 2{,}000 = 280$ TFLOPs. An H100 at 495 TFLOPS dense needs $\sim$566 ms. If 4 users send prompts simultaneously, the 4th waits over 1.5 seconds before prefill even begins.
>
> **Answer**: Prefill starvation. The scheduler must chunk or defer prefills to prevent blocking the decode phase, but aggressive deferral under high load creates a queue that destroys TTFT. The fix is chunked prefill with priority scheduling, but this requires over-provisioning compute by $\sim$30% to absorb the prefill bursts without queuing.

This is a *mastery* question. The candidate must recall hardware specs (H100 TFLOPS), analyze the scheduling conflict (prefill blocks decode), design a solution (chunked prefill with priority), and execute the napkin math to size the required over-provisioning. All four skills are exercised in a single question.

Timeless principle: In multiplexed serving, prefill and decode compete for the same compute; any scheduling policy must bound prefill latency to prevent decode starvation.

11.5 Cross-Track Comparison: Quantization

The same topic, quantization, manifests differently across physics regimes, illustrating why the track axis is essential:

- **Cloud** (L4, evaluation). "GPTQ vs. AWQ for serving a 70B model on 2× H100s. Which quantization scheme yields higher throughput at FP16-equivalent perplexity? Justify with arithmetic intensity analysis." The candidate evaluates two compression strategies against datacenter-scale metrics (tokens/second/dollar).
- **Edge** (L3, fluency). "Quantize a YOLOv8 detection model from FP32 to INT8 for Orin deployment. Calculate the memory savings and estimate the latency improvement given Orin's INT8 TOPS vs. FP32 TFLOPS." The candidate performs napkin math within a thermal and real-time envelope.
- **TinyML** (L5, specification). "Your 256 KB MCU must run a keyword spotting model. The FP32 model is 800 KB. Specify a quantization strategy (INT8, INT4, or binary) that fits in SRAM while maintaining >90% accuracy. Justify each precision choice for weights vs. activations." The candidate designs a mixed-precision scheme under extreme memory constraints.

All three questions test quantization knowledge, but the physics regime determines the constraints, the formulas, and the engineering tradeoffs. A candidate who can answer the cloud question may struggle with the TinyML variant, because the reasoning transfers only partially across regimes.

These five questions demonstrate how the four-axis classification captures qualitatively different cognitive acts across the ML systems engineering domain. Fluency tests whether you can *compute* a number. Diagnosis tests whether you can *explain* an observation. Specification tests whether you can *translate constraints into architecture*. Mastery tests whether you can *synthesize* a complete solution. The cross-track comparison shows that the same topic demands different reasoning in different physics regimes.

Future work includes questions that provide partial information, requiring candidates to identify what additional data they need — a skill that tests meta-cognitive reasoning about systems diagnostics. Such incomplete-information questions would assess whether candidates know what they do not know, a hallmark of Staff-level engineering judgment.

12 Practical Applications

The zone model translates directly into interview loop design. A typical four-round interview should cover at least four zones to assess breadth of systems reasoning:

- **Phone screen.** Fluency questions. Can the candidate perform napkin math from memory? This is the fastest signal for mechanical sympathy.
- **System design.** Specification or Design questions. Can the candidate architect a system under quantitative constraints?
- **Debugging.** Diagnosis questions. Can the candidate root-cause a failure from observable symptoms and hardware specifications?
- **Architecture review.** Evaluation or Mastery questions. Can the candidate analyze tradeoffs and justify design decisions with quantitative reasoning?

A "strong hire" signal at Staff level is consistent strength in compound zones, especially diagnosis and evaluation. A "no hire" signal is strong recall but weakness across all compound zones — the profile of a memorizer rather than a reasoner. No interview loop should test only pure

zones, as these fail to exercise the integrative reasoning that distinguishes Staff-level engineers from mid-level ones.

Not all 11 zones are equally important for interview design. The five compound zones (diagnosis, specification, fluency, evaluation, and optimization) and mastery carry the most signal for Staff+ assessment. The four pure zones (recall, analyze, design, quantify) serve primarily as corpus organization categories and warm-up question pools; they are the building blocks from which compound zones are constructed, not assessment targets in their own right. Realization (design ∩ quantify), while theoretically defined, appears infrequently in practice because most sizing questions also require recall, making them fluency or specification questions instead.

13 Related Work

STAFFML draws on three distinct bodies of work: interview preparation platforms that define the practical landscape, ML systems research that provides the technical content, and educational measurement theory that grounds the assessment methodology. This section surveys each area and positions STAFFML relative to the prior art.

13.1 Interview Platforms

LeetCode (LeetCode Inc., 2024) hosts 2,800+ algorithmic coding problems with community-driven difficulty calibration and is the dominant platform for software engineering interview preparation. NeetCode (NeetCode, 2024) curates 150 problems organized by algorithmic pattern with explicit difficulty progression. Grind 75 (Yang, 2024) ranks 169 problems by interview frequency and pattern coverage. All three platforms excel at algorithmic coding assessment but provide no coverage of systems reasoning, hardware constraints, or napkin math — the cognitive skills that ML systems interviews demand.

Huyen's *Designing Machine Learning Systems* (Huyen, 2022) provides comprehensive conceptual coverage of ML system design, including data engineering, model serving, and monitoring. The text is widely used for interview preparation but does not include quantitative practice problems or a structured difficulty progression. Candidates who study the book understand *what* a feature store is but cannot *calculate* the memory bandwidth required to serve a 70B model at target throughput.

Table 9 summarizes the positioning. STAFFML addresses the gap between these extremes: the quantitative rigor of LeetCode-style practice (every question has a definite numerical answer) combined with the systems focus of conceptual interview guides (every question is grounded in real hardware constraints). The four-axis classification adds structured difficulty progression and cognitive-mode coverage that no existing platform provides.

Table 9. Interview preparation landscape. Existing platforms optimize for algorithmic coding or conceptual systems knowledge; STAFFML is the first to combine quantitative rigor with ML systems focus and a structured cognitive taxonomy.

Platform	Items	Systems	Quantitative	HW-Grounded	Cognitive Model
LeetCode	2,800+	—	✓	—	Easy / Med / Hard
NeetCode	150	—	✓	—	Pattern-based
Grind 75	169	—	✓	—	Frequency rank
Huyen (DMLS)	Conceptual	✓	—	—	—
STAFFML	**9,199**	✓	✓	✓	**11 zones × 6 levels**

13.2 ML Systems Research

The technical content of STAFFML questions is grounded in seminal ML systems research. Megatron-LM (Shoeybi et al., 2019) introduced tensor parallelism for training large language models, providing the foundation for questions about parallelism strategies and communication overhead. GPipe (Huang et al., 2019) established pipeline parallelism and the bubble fraction analysis that appears in realization-zone questions. ZeRO (Rajbhandari et al., 2020) and DeepSpeed (Rasley et al., 2020) eliminated memory redundancy in data parallelism, motivating questions about optimizer state partitioning and memory budgeting.

FlashAttention (Dao et al., 2022) demonstrated IO-aware algorithm design, reducing memory reads/writes from quadratic to linear in sequence length, and provides the canonical example of how understanding the memory hierarchy yields algorithmic improvements invisible to complexity analysis alone. vLLM (Kwon et al., 2023) introduced PagedAttention for KV-cache management, directly inspiring our fluency and diagnosis questions about memory fragmentation and serving throughput. Orca (Yu et al., 2022) pioneered continuous batching (iteration-level scheduling), which motivates our mastery-zone questions about prefill starvation and scheduling tradeoffs.

MLPerf (Mattson et al., 2020; Reddi et al., 2020) established standardized benchmarks for measuring ML *system* performance: training time, inference latency, and throughput across hardware platforms. The distinction between MLPerf and STAFFML is important. MLPerf measures how well systems perform; STAFFML measures how well *humans* reason about systems performance. MLPerf's measurement methodology (standardized workloads, reproducible configurations, hardware-specific baselines) informs the quantitative grounding of STAFFML questions, and several questions directly reference MLPerf benchmarking scenarios. However, the assessment targets are fundamentally different — MLPerf evaluates hardware and software implementations, while STAFFML evaluates human engineering judgment about those implementations.

These systems are not merely cited; they are the *source material* for STAFFML questions (Table 10). The questions

Table 10. Source material mapping. Each seminal system provides the quantitative content for questions in specific zones. The mapping illustrates how STAFFML operationalizes research into assessment.

System	Concept Tested	Primary Zones	Example Task
Megatron-LM	Tensor parallelism	Realization, Fluency	Size the TP degree for 70B on 8×H100
GPipe	Pipeline bubbles	Realization, Optimization	Calculate bubble fraction for k micro-batches
ZeRO / DeepSpeed	Memory partitioning	Fluency, Diagnosis	Budget optimizer state across N GPUs
FlashAttention	IO-aware attention	Optimization, Quantify	Derive memory reads: standard vs. tiled
vLLM	KV-cache paging	Diagnosis, Fluency	Calculate fragmentation at 128K context
Orca	Continuous batching	Mastery, Evaluation	Diagnose prefill starvation under load
MLPerf	System benchmarking	Quantify, Analyze	Interpret throughput vs. latency tradeoff

test whether candidates understand the systems principles these papers discovered.

13.3 Educational Measurement

The assessment design methodology draws on three traditions. Bloom's revised taxonomy (Anderson et al., 2001; Bloom et al., 1956) provides the six-level cognitive framework that maps to our mastery levels, validated by decades of educational research as a reliable predictor of assessment difficulty (Webb, 1997; Biggs and Collis, 1982). Evidence-Centered Design (Mislevy et al., 2003) provides the three-component assessment architecture (claim, evidence, task features) that structures our construction pipeline. Automatic Item Generation (Gierl and Haladyna, 2013; Various Authors, 2024) provides the theoretical foundation for template-based question generation, extended here with hardware-grounded construction and multi-model verification.

The topic taxonomy follows SKOS (W3C, 2009) for knowledge organization, faceted classification principles (Hjørland, 2013) for orthogonal axis design, and test blueprinting methodology (Millman and Greene, 1989; Case and Swanson, 2002) for coverage validation. Item Response Theory (Lord, 1980; Hambleton et al., 1991) provides the psychometric framework for future difficulty calibration once sufficient response data ($\geq$30 responses per item) is collected.

The TinyTorch educational framework (Reddi, 2026c) provides a complementary pedagogical approach: where STAFFML assesses whether engineers *have* ML systems competencies, TinyTorch teaches those competencies through a build-from-scratch curriculum spanning 20 modules.

14 Limitations

We identify four limitations of the current work.

No empirical validation. The question bank has been editorially validated through multi-model cross-review but has not yet been psychometrically validated through user studies. Correlating zone-level performance with engineering seniority is an important next step. Until response data is collected, zone classifications, level assignments, and difficulty orderings reflect expert judgment rather than empirical measurement of candidate performance.

Static questions with chain-based progression. While individual questions are static text, the depth chain structure enables adaptive progression: a candidate who answers an L3 question correctly is presented with the L4 question on the same topic. This simulates some of the adaptivity of live interviews, though it does not replicate the free-form dialogue of whiteboard sessions. The chains (Section 3.5) approximate the probing behavior of experienced interviewers.

Hardware vendor concentration. The corpus draws predominantly on NVIDIA hardware specifications (H100, A100, Orin), reflecting current industry adoption. Candidates with primary experience on AMD, Intel, or Google TPU hardware may find the specific constants unfamiliar, though the reasoning patterns (roofline analysis, memory budgeting, power-performance tradeoffs) are vendor-neutral. Future versions will include multi-vendor question variants. For assessment contexts, we recommend providing a hardware specification reference sheet alongside the questions, ensuring that recall of vendor-specific numbers does not gate access to the reasoning being tested.

Hardware constant maintenance. Every question is grounded in specific hardware specifications (H100: 80 GB HBM3, 3.35 TB/s, 495 TFLOPS FP16 dense; Orin: 275 TOPS INT8). These numbers are correct as of the current accelerator generation but require updates as new hardware ships. The shared physics engine centralizes constants to make updates tractable. Changing one constant propagates to all affected questions, and the invariant system flags any question whose napkin math references outdated specifications. The methodology is designed to be timeless even as specific numbers evolve.

15 Conclusion and Future Work

STAFFML introduces a structured approach to assessing ML systems engineering expertise. The core contribution is the *ikigai competency model*, which decomposes systems expertise into four fundamental skills (recall, analyze, design, quantify) whose pairwise intersections produce eleven cognitive zones that capture qualitatively different acts of engineering reasoning. This model, combined with questions that require napkin math against real hardware specifications, provides a vocabulary for what it means to "know" ML systems engineering at each level of professional seniority.

The four-axis classification framework (topic, zone, track, level) operationalizes this competency model into a practical assessment system. The topic taxonomy organizes 87 concepts with prerequisite relationships that enable personalized study paths. The track axis grounds every

question in a specific physics regime, ensuring that assessment reflects the real constraints of cloud, edge, mobile, and TinyML deployment. The level axis maps to both Bloom's cognitive taxonomy and industry engineering ladders, making STAFFML actionable for candidates preparing for specific roles and hiring managers calibrating interview loops.

Empirical validation through user studies is the most important next step. Looking ahead, we plan to implement Item Response Theory (IRT) calibration once sufficient responses are collected, assigning empirical difficulty and discrimination parameters to each item. To expand the corpus, a contribution pipeline with automated schema validation will enable industry practitioners to submit questions grounded in real production failure modes. We also envision adaptive question selection that adjusts to individual performance profiles, targeting the specific zones where a candidate is weakest. Finally, to eliminate the reliance on probabilistic LLM math verification entirely, the next evolution of STAFFML is *executable napkin math*: formalizing every question's calculations into symbolic expressions evaluated directly by the deterministic MLSysIM physics engine. The prerequisite graph will also enable graph-guided study paths that sequence topics based on dependency structure rather than arbitrary ordering.

The corpus, schema, and quality infrastructure are available as a companion to the *Machine Learning Systems* textbook (Reddi, 2026a)[6].

References

Lorin W. Anderson, David R. Krathwohl, Peter W. Airasian, Kathleen A. Cruikshank, Richard E. Mayer, Paul R. Pintrich, James Raths, and Merlin C. Wittrock. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, New York, 2001. The revised Bloom's taxonomy: Remember, Understand, Apply, Analyze, Evaluate, Create.

AnKing Team. AnKing step deck — community quality control, 2024. URL https://www.ankingmed.com. Large-scale Anki deck with community-driven dedup and quality flags.

John B. Biggs and Kevin F. Collis. *Evaluating the Quality of Learning: The SOLO Taxonomy (Structure of the Observed Learning Outcome)*. Academic Press, New York, 1982. SOLO taxonomy: prestructural, unistructural, multistructural, relational, extended abstract.

Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. *Taxonomy of Educational Objectives: The Classification of Educational Goals*. David McKay Company, New York, 1956. Original Bloom's taxonomy. Revised by Anderson & Krathwohl (2001).

Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, pages 21–29. IEEE, 1997. doi: 10.1109/SEQUEN.1997.666900. MinHash for near-duplicate detection at web scale.

Susan M. Case and David B. Swanson. *Constructing Written Test Questions for the Basic and Clinical Sciences*. National Board of Medical Examiners, Philadelphia, PA, 3rd edition, 2002. USMLE test blueprinting with 4+ independent classification axes per item.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. IO-aware attention algorithm that reduces memory reads/writes from quadratic to linear in sequence length.

Mark J. Gierl and Thomas M. Haladyna. *Automatic Item Generation: Theory and Practice*. Routledge, New York, 2013. doi: 10.4324/9780203803912. Foundational text on template-based assessment item generation.

Ronald K. Hambleton, Hariharan Swaminathan, and H. Jane Rogers. *Fundamentals of Item Response Theory*. Sage Publications, Newbury Park, CA, 1991. Practical guide to IRT. 30+ responses needed for stable calibration.

Birger Hjørland. Facet analysis: The logical approach to knowledge organization. *Information Processing & Management*, 49(2):545–557, 2013. Faceted classification: independent orthogonal axes rather than a single hierarchy.

Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. Pipeline parallelism for training models that exceed single-device memory.

Chip Huyen. *Designing Machine Learning Systems*. O'Reilly Media, Sebastopol, CA, 2022. ISBN 978-1098107963. ML systems design textbook focused on production deployment.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao

[6] https://staffml.dev

Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2023. doi: 10.1145/3600006.3613165. vLLM: virtual memory paging for KV-cache reduces fragmentation and enables higher throughput.

LeetCode Inc. LeetCode — online coding platform, 2024. URL https://leetcode.com. 2,869+ validated coding problems with community-driven difficulty calibration.

Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. MCUNet: Tiny deep learning on IoT devices. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1980. Foundational IRT text. b-parameter (difficulty), a-parameter (discrimination).

Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St John, Carole-Jean Wu, Lingjie Xu, Cliff Young, and Matei Zaharia. MLPerf Training Benchmark. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 336–349, 2020. The original MLPerf Training benchmark establishing standardized ML system measurement.

Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995. doi: 10.1037/0003-066X.50.9.741. Unified validity framework: content, construct, consequential validity.

Jason Millman and Jennifer Greene. Test blueprints. In Robert L. Linn, editor, *Educational Measurement*. American Council on Education / Macmillan, New York, 3rd edition, 1989. Test blueprint / table of specifications for coverage validation.

Robert J. Mislevy, Russell G. Almond, and Janice F. Lukas. A brief introduction to evidence-centered design. In *Research Report*. Educational Testing Service, Princeton, NJ, 2003. ECD framework: claims, evidence, task features for assessment design.

NeetCode. NeetCode 150 — curated coding interview problems, 2024. URL https://neetcode.io/practice. 150 problems organized by pattern with difficulty progression.

OpenAI. Gpt-4 technical report. Technical report, 2023.

Samyam Rajbhandari, Jeff Rasley, Olatunji Rber, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2020. doi: 10.1109/SC41405.2020.00024. ZeRO optimizer partitioning eliminates memory redundancy in data parallelism.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506. ACM, 2020. doi: 10.1145/3394486.3406703. Training system combining ZeRO, pipeline parallelism, and mixed precision for 100B+ parameter models.

Vijay Janapa Reddi. CS249r: Tiny machine learning — harvard university, 2024. URL https://sites.google.com/g.harvard.edu/cs249-tinyml-2024. Graduate course on ML systems spanning cloud to embedded deployment.

Vijay Janapa Reddi. *Machine Learning Systems*. MIT Press, 2026a. URL https://mlsysbook.ai. Two-volume textbook on ML systems following the Hennessy & Patterson pedagogical model. Volume I covers single-machine systems; Volume II covers distributed systems at scale.

Vijay Janapa Reddi. MLSysIM: A physics simulator for machine learning systems, 2026b. URL https://mlsysbook.ai/mlsysim. Companion simulator for the ML Systems textbook. Provides a shared physics engine with hardware constants, roofline analysis, and quantitative verification of systems reasoning.

Vijay Janapa Reddi. Tinytorch: Building machine learning systems from first principles. *arXiv preprint arXiv:2601.19107*, 2026c. Educational ML framework with 20 modules teaching ML as systems engineering from first principles.

Vijay Janapa Reddi. *Machine Learning Systems at Scale*. MIT Press, 2026a. URL https://mlsysbook.ai. Volume II of the Machine Learning Systems textbook.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu,

Brian Anderson, Maximilien Breesmoer, Mark Kanber, Qing-Hua Chou, et al. MLPerf Inference Benchmark. In *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, pages 446–459. IEEE, 2020. doi: 10.1109/ISCA45697.2020.00045. Standardized benchmarks for ML system performance across inference scenarios.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. Tensor parallelism for training large language models across GPUs.

Dagobert Soergel. *Organizing Information: Principles of Data Base and Retrieval Systems*. Academic Press, Orlando, FL, 1985. User warrant vs literary warrant in knowledge organization.

Martin Thompson. Mechanical sympathy, 2011. URL https://mechanical-sympathy.blogspot.com/. Blog. The term describes software that works with the hardware rather than against it, inspired by racing driver Jackie Stewart's philosophy.

Various Authors. Leveraging large language models for automatic item generation: A multi-agent approach. *PMC / Frontiers in Education*, 2024. LLM-generated items achieve comparable psychometric properties to expert-authored items with multi-agent review. PMC10700404.

W3C. SKOS simple knowledge organization system reference. W3c recommendation, World Wide Web Consortium, 2009. URL https://www.w3.org/TR/skos-reference/. Standard for broader/narrower/related concept relationships.

Norman L. Webb. Criteria for alignment of expectations and assessments in mathematics and science education. *Research Monograph No. 6*, 1997. Depth of Knowledge (DOK) framework: four levels of cognitive complexity for assessment alignment.

Grant Wiggins and Jay McTighe. *Understanding by Design*. Association for Supervision and Curriculum Development, Alexandria, VA, 2nd edition, 2005. Backward design methodology: desired results, acceptable evidence, then learning plan.

Yangshun Yang. Grind 75 — dynamically ranked interview problems, 2024. URL https://www.techinterviewhandbook.org/grind75. 169 problems ranked by frequency, pattern coverage, and difficulty.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538. USENIX Association, 2022. Iteration-level scheduling (continuous batching) for transformer serving, eliminating padding waste.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.